

# Graph Neural Networks for Reliability and Dependability Analysis in Complex Distributed Systems based on Publish–Subscribe Architecture

---

## Abstract

Publish–subscribe middleware decouples producers and consumers, improving scalability but obscuring the dependency chains along which one component’s failure cascades. Runtime telemetry is absent before deployment and code-level static analysis is blind to system-level topology, so identifying *which* components are critical — and *why* — remains difficult. We present **Software-as-a-Graph (SaG)**, a pre-deployment **Static System Analysis** framework that models a pub-sub system as a typed, weighted multigraph and derives logical dependencies through typed projection rules. On this representation we train a relation-specific **Heterogeneous Graph Transformer** to forecast cascading failure impact, and pair it with an interpretable score decomposing criticality into Reliability, Maintainability, Availability and Vulnerability (RMAV). Both are compared against discrete-event cascade simulators under an input–label independence guarantee. Across seven synthetic topologies and three graphs transcribed from open-source architectures we report four results. **(1)** Typed learning leads the strongest non-learning baseline on rank correlation ( $\rho = 0.730$  vs  $0.595$ ), though unconfirmed by a paired test; the critical-set advantage is more robust ( $F_1@K = 0.465$  vs  $0.308$ ). **(2)** Attribution earns its place as explanation rather than accuracy: equal dimension weights outperform the calibrated weighting. **(3)** The two cascade oracles agree only weakly ( $\rho = 0.394$ ), bounding construct validity. **(4)** Edge criticality is measured by removal rather than inferred, showing most links replaceable. Finally, SaG operates as a blocking CI/CD gate, evaluating a candidate topology in under a minute, though the anti-pattern catalog’s agreement with the cascade oracle is modest, and the gate is currently absolute rather than delta-aware.

*Keywords:* publish–subscribe middleware, architectural dependability, cascading failure, heterogeneous graph neural networks, static system analysis, pre-deployment verification, quality attributes, CI/CD quality gate

---

## 1. Introduction

### 1.1. Motivation

The publish–subscribe (pub-sub) paradigm has become a backbone communication abstraction for large-scale distributed systems, underpinning cyber-physical, cloud-native, robotics, and Internet-of-Things

architectures. Its appeal is decoupling: producers and consumers are separated in time, space, and synchronization, so components can be added, removed, or scaled without direct knowledge of one another [1]. Industry standards such as the Data Distribution Service (DDS) and MQTT formalize this model and expose deployment-time choices — topics, brokers, reliability, durability, and other Quality-of-Service (QoS) policies — that materially shape how the system behaves under stress [2, 3].

The same decoupling that makes pub-sub flexible also obscures the dependency structure an engineer must reason about when a component fails. There are no explicit caller–callee edges: an application that publishes to a topic has no static link to the applications that subscribe to it, even though those subscribers are wholly dependent on it for data. Failures do not propagate along a call graph but along *derived* paths — through shared topics and brokers, through colocated deployment nodes, and, distinctively, through shared libraries whose failure strikes every consumer *simultaneously* rather than sequentially. A raw architecture diagram does not reveal these chains, and the components whose failure would be most damaging are frequently not the ones a diagram makes look important.

Crucially, the moment at which this reasoning is most valuable is *before* deployment. Architectural hardening — replication, isolation, failover, additional monitoring — is cheapest and least disruptive while the system is still a design, and prohibitively expensive once it is in production. Yet pre-deployment is precisely when no runtime telemetry exists to identify weak points empirically. An engineer must therefore answer a hard question from the architecture alone: *which components are critical, and why?*

Beyond operational dependability, pre-deployment failure prevention is directly tied to **software sustainability and infrastructure resource efficiency**. Uncontained cascading failures in modern distributed systems — ranging from cyber-physical fleets to cloud-native microservices — trigger emergency server re-provisioning, redundant message re-transmission storms, and high-frequency failover loops that consume substantial electrical power and compute capacity. Preventing these architectural failure cascades at design time eliminates post-deployment energy waste and unnecessary infrastructure expenditure, directly supporting sustainable computing practices.

## 1.2. The Architecture-Code Gap and Problem Statement

We address pre-deployment criticality analysis for pub-sub middleware as two coupled sub-problems. Given only an architectural description of the system — its applications, libraries, topics, brokers, deployment nodes, and the QoS policies on its communication — we seek to:

1. **Quality attribution.** Assign each component an interpretable measure of *how* and *why* it is critical, grounded in the **ISO/IEC 25019:2023 Quality-in-Use** standard (Beneficialness, Freedom from Risk, Acceptability) and decomposed along the quality dimensions an engineer would act on, so that the result directs a specific remediation rather than a generic warning.

2. **Failure-impact analysis.** Predict the cascade impact of each component’s failure — the extent to which the rest of the system becomes unreachable or impaired — and identify the components that should be hardened first.

Both must be computed without runtime data, and both must remain *explainable*: a single opaque criticality number is of limited use to an architect who has to choose between competing interventions under a fixed budget.

Historically, static verification has operated primarily at the source-code level. However, a major “**Architecture-Code Gap**” exists: a software system can have perfectly clean source code in every component (earning top scores on code-level tools), yet remain highly fragile. If the deployment topology contains a Single Point of Failure (SPOF) or a mismatched QoS contract, a single component crash can cascade and collapse the entire system. Bridging this gap requires shifting structural verification “left” into the continuous integration and delivery (CI/CD) pipeline.

### 1.3. Limitations of Existing Approaches

Three strands of prior work bear on this problem, and each leaves a gap.

**Static Code Analysis (SCA).** Platforms such as SonarQube evaluate code cleanliness, cyclomatic complexity, and LCOM (Lack of Cohesion of Methods) inside individual modules. While highly effective for intra-component quality, they are entirely blind to inter-component topologies and dynamic middleware cascades.

**Runtime Dependability and Chaos Engineering.** A large body of work hardens pub-sub systems through runtime fault tolerance, replication, and chaos injection (e.g., Chaos Monkey). These techniques are valuable but assume a *running* staging or production system; they do not answer which components a design should protect before it is deployed, and injecting failures at runtime carries operational risk.

**Topology-Only and Homogeneous Learning-Based Centrality.** Classical network-science metrics collapse a component’s risk into a single scalar that conflates distinct failure mechanisms (SPOFs vs. cascade hubs), while homogeneous graph neural networks collapse typed semantics (applications, topics, brokers) into flattened views, leading to representation collapse.

No existing approach offers an *interpretable, multi-dimensional, pre-deployment* attribution over the *typed* pub-sub graph, coupled to code-level SCA metrics, heterogeneous-GNN impact prediction, and automated CI/CD gating. That is the gap this paper fills.

### 1.4. Our Approach

We present **Software-as-a-Graph (SaG)**, a pre-deployment **Static System Analysis (SSA)** framework. SaG models a pub-sub system as a typed, weighted, directed multigraph over five node types (applications, libraries, topics, brokers, nodes) and derives logical `DEPENDS_ON` dependencies through typed

projection rules. Crucially, SaG ingests code-level SCA metrics as vertex attributes and performs **multi-dimensional quality attribution**, decomposing criticality into orthogonal Reliability, Maintainability, Availability, and Vulnerability (RMAV) dimensions under a stated weighting audited for Analytic Hierarchy Process (AHP) consistency (Section 4.3).

SaG then performs **failure-impact analysis**, predicting cascade impact  $I(v)$  with two predictors: the multi-dimensional composite  $Q(v)$  and a learned **Heterogeneous Graph Transformer (HGL)**. We evaluate the learned predictor in two variants — QoS-masked (HGL) and QoS-encoded (*HGL-QoS*) — to isolate what explicit QoS contract features contribute; Section 8.3 reports that contribution as a null, so every headline figure in this paper is the QoS-masked HGL, and we name the variants separately throughout rather than presenting the QoS-encoded model as the framework’s predictor. Both are validated against a discrete-event simulator under an **input-label independence guarantee**. Finally, a **prescriptive remediation** stage generates topology-level hardening edits and verifies them on counterfactual graphs in-memory.

To make SSA continuous, SaG integrates directly into CI/CD pipelines as a blocking gate: a dedicated CLI script runs the anti-pattern catalog against the candidate topology in seconds and fails the build (exit code 2) when it finds CRITICAL or HIGH severity structural anomalies (Section 6.6). The gate is currently absolute rather than delta-aware — it evaluates the full finding set on every run rather than diffing against a merge-base baseline — a limitation Section 6.6 and Section 9.3 discuss and scope as future work.

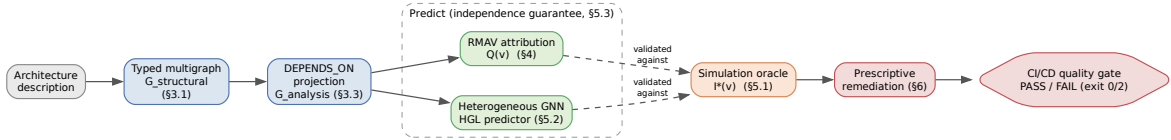


Figure 1: end-to-end SaG pipeline — architecture description → typed multigraph → `DEPENDS_ON` projection → the two predictor paths and the simulation oracle path, with the independence boundary between them marked → remediation and the CI/CD gate.

Concretely, the paper is organized around four research questions:

**RQ1.** For pre-deployment criticality prediction, *where* does typed graph learning improve on non-learning structural baselines — in recovering the full impact ordering, in identifying the critical set, or both — and does that answer differ in-distribution versus on unseen architectures?

**RQ2.** What does taking node and edge type seriously expose that a single-score topological centrality misses, and what does it fail to expose?

**RQ3.** How does explicit multi-attribute QoS contract feature injection affect in-distribution convergence versus out-of-distribution Leave-One-Scenario-Out (LOSO) generalizability?

**RQ4.** What is the feasibility and performance overhead of deploying the graph-based analyzer as a blocking Quality Gate in continuous integration/delivery (CI/CD) pipelines?

**RQ5.** Does the framework’s predictive ranking transfer to architectures it did not generate — that is, to systems specified independently of our topology generator?

RQ1 is deliberately phrased as *where* rather than *whether*: the answer turns out to depend on which metric the question is asked about, and a formulation that admits only “learning is / is not required” would have obscured that (Section 8.1).

RQ1, RQ2, and RQ3 are answered on the synthetic scenario suite (Section 8.1–Section 8.3); RQ4 evaluates gating feasibility and performance (Section 8.4); and RQ5 is answered on three graphs transcribed from real open-source architectures (Section 8.5). RQ5 carries the paper’s external validity and is, correspondingly, the question our evidence answers most weakly — Section 8.5 states in full what three hand-built graphs with simulated ground truth can and cannot establish.

### 1.5. Contributions

This paper makes the following contributions:

1. **A typed graph model with hierarchical SCA metric integration.** We define the SaG multigraph and the RMAV decomposition, which propagates code-level quality metrics (SonarQube `cm_*` fields) into global system criticality scores (Section 3, Section 4).
2. **A scope condition on where graph learning pays for pub-sub criticality.** Under a single evaluation contract applied to every predictor (Section 7.3), typed learning leads the strongest training-free baseline on both ranking ( $\rho = 0.608$  vs 0.521 out of distribution) and critical-set identification ( $F_1@K = 0.465$  vs 0.308) — after repairing that baseline, which was silently computing unweighted betweenness on every scenario (Section 8.1). We report the repair because a baseline accidentally identical to the one it should improve on inflates any margin measured against it. We also report what the two halves of that claim are worth: the critical-set advantage holds under every protocol and every version of the apparatus, whereas the ranking margin fails a paired significance test in-distribution and rests on an unretained artifact out of distribution (Section 8.1, Section 9.2). The contribution is the scope condition, not the win.
3. **Multi-dimensional criticality attribution, positioned as explanation rather than accuracy.** RMAV decomposes criticality into four dimensions with distinct remediation owners, so a diagnostic is traceable to an action. A shrinkage sweep shows the dimension weighting does *not* improve ranking accuracy over equal weights (Section 8.3); we report this and scope the contribution to attribution accordingly (Section 4, Section 8.3).

- 128 **4. Relationship criticality as a first-class measure, and measured edge ground truth.** We  
129 give inter-component dependencies the same four-dimensional attribution as components (Section 4.7),  
130 so that the partial-outage case — one link down, both endpoints healthy — is scored rather than  
131 inferred from endpoint scores. Separately, we obtain edge ground truth by simulating removal of each  
132 candidate relationship rather than projecting node labels through a heuristic multiplier, finding that  
133 most individual links are replaceable and exposing a class of structurally non-redundant edges the  
134 cascade model cannot express (Section 8.2). We are explicit that these two are computed over different  
135 edge populations, so the second does not validate the first (Section 4.7, Section 9.3).
- 136 **5. An automated CI/CD quality gate.** We formulate a build-blocking gate that evaluates system-level  
137 structural risk statically and executes in well under a minute on every scenario in our corpus (Section 6,  
138 Section 8.4). The gate is absolute rather than delta-aware in the current implementation; Section 6.6  
139 and Section 9.3 scope delta-awareness and a waiver register as design work this contribution does not  
140 yet include.
- 141 **6. A prescriptive remediation stage with per-edit counterfactual verification.** We formalise  
142 a Generate→Verify procedure in which every candidate edit is simulated in isolation and admitted  
143 only if it improves impact by more than the simulator’s seed noise, at every propagation threshold  
144 (Section 6.4, Section 6.7).
- 145 **7. An account of the evaluation methodology itself.** We document two defects in our own harness  
146 — non-matching evaluation populations across predictor families, and a stale-checkpoint path that  
147 silently skipped training — that produced published-looking numbers of the wrong sign, together with  
148 the contract that prevents each (Section 7.3, Section 9.2). We report this because both failure modes  
149 are invisible in the output and, we suspect, not unique to this study.
- 150 **8. Empirical real-world validation on open-source software architectures.** We demonstrate  
151 SaG’s external validity on three authentic real-world software graphs — the Autoware.universe ROS 2  
152 autonomous driving platform, a production Cloud-Native Microservices mesh, and the Train-Ticket  
153 railway-booking mesh (Section 7.1, Section 8.5) — achieving high mean rank agreement over five seeds  
154 ( $\rho = 0.688, 0.778, 0.759$ ) and up to  $F_1@K = 1.000$  on two of the three, though 5 of the 15 total gate  
155 checks fail across the three graphs — all three fail SPOF-F1, and Train-Ticket is the only one for  
156 which SPOF-F1 is the *sole* failure — and  $F_1@K = 1.000$  is partly a tie-breaking artifact where the  
157 number of genuinely non-zero-impact components is smaller than  $K$  (Section 8.5). What the three  
158 cases jointly support is that SaG’s predictive ranking generalizes beyond the synthetic generator to  
159 independently-sourced architectures across two paradigms, not an unqualified success on production  
160 software systems.

### 1.6. Relationship to the Authors' Prior Work

This work extends the authors' earlier structural baseline of the framework — multi-layer graph dependency analysis — introduced in prior work [4]. The present paper consolidates that structural foundation with the heterogeneous graph neural network predictor, multi-dimensional quality attribution (Section 4), SCA metric integration, and CI/CD gating and remediation (Section 6) into a single, self-contained submission targeted at this special issue's focus on AI for reliability and dependability analysis; no companion manuscript reporting the heterogeneous-GNN predictor is submitted or under review in parallel with this paper.

### 1.7. Organization

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the Software-as-a-Graph model. Section 4 presents multi-dimensional quality attribution, and Section 5 presents failure-impact analysis. Section 6 introduces prescriptive remediation and CI/CD quality gating. Section 7 describes the experimental setup; Section 8 reports the synthetic-suite and gating results (RQ1–RQ4); and Section 9 discusses the findings, threats to validity, and conclusions.

---

## 2. Related Work

This paper draws on, and contributes to, several established lines of research: publish–subscribe dependability, static analysis techniques, pre-deployment system verification, structural criticality, and multi-criteria quality scoring.

### 2.1. Publish–Subscribe Middleware and Dependability

The pub-sub paradigm is a foundational communication abstraction for large-scale distributed systems, valued for decoupling producers and consumers in time, space, and synchronization [1]. Content-based and brokered overlays extend this with flexible event routing and subscription matching, and standards such as DDS and MQTT formalize deployment-time choices, alongside log-structured brokers such as Kafka [5] and robotics middleware such as ROS 2 [6], — topics, brokers, reliability, durability, and other QoS policies — that govern runtime behavior [2, 3]. These mechanisms enable cyber-physical, cloud, IoT, and robotics architectures, but they also make failure propagation difficult to reason about from direct communication edges alone.

Research on pub-sub dependability has accordingly emphasized runtime fault tolerance, reliable event dissemination, replication, and recovery. These approaches improve a system's resilience while it is *running*: they assume observable behavior and react to or mask faults as they occur. Our concern is complementary and earlier in the lifecycle — estimating, from an architectural model that enumerates applications, libraries,

192 topics, brokers, and QoS policies, which components would have the greatest downstream impact if they  
193 failed, so that the design can be hardened before any system is deployed.

## 194 2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

195 Static verification typically operates at the source-code level. Static Code Analysis (SCA) tools, exemplified  
196 by SonarQube, checkstyle, and FindBugs, parse source files into Abstract Syntax Trees (ASTs) to compute  
197 complexity [7], code duplication, and modular metrics such as LCOM (Lack of Cohesion of Methods)  
198 [8, 9]. While SCA is essential for locating intra-component defects and technical debt, it is blind to the  
199 inter-component topology.

200 Static System Analysis (SSA) addresses this “Architecture-Code Gap.” SSA models the system as a global  
201 graph of communicating components, middleware routers, and hardware hosts. Rather than replacing SCA,  
202 SSA ingests code-level metrics as node properties (e.g., LCOM, cyclomatic complexity) and propagates them  
203 through the inter-component dependency topology. This allows architects to evaluate how code-level fragility  
204 (e.g., a highly complex class inside an application) combines with structural fragility (e.g., the application  
205 being a single point of failure) to create systemic risks.

## 206 2.3. Continuous Pre-Deployment Verification and Gating

207 A common way to verify system resilience is dynamic testing, particularly Chaos Engineering [10],  
208 popularised by Netflix’s Chaos Monkey, which injects faults into live staging or production clusters. While  
209 chaos testing evaluates real operational environments, doing so carries risk and occurs late in the lifecycle.

210 Continuous pre-deployment verification shifts this analysis left, integrating it into CI/CD pipelines [11, 12].  
211 In this paradigm, the system architecture is defined as “Architecture-as-Code” (AaC) via configuration  
212 descriptors (Docker Compose, Kubernetes manifests, Helm charts). SSA tools run automatically on every  
213 pull request, parsing the configuration descriptors to generate a counterfactual topology graph, and block  
214 the build (exiting with non-zero status) when a change introduces critical architectural smells (like SPOFs  
215 or QoS mismatches) or exceeds failure-propagation thresholds. Mature code-level gates follow the same  
216 discipline: SonarQube’s default “Clean as You Code” quality gate evaluates *new* code against the merge  
217 base rather than failing builds on the accumulated state of the whole codebase, and pairs the gate with an  
218 explicit won’t-fix/false-positive marking workflow [13]. Our system-level gate adopts the analogous semantics  
219 — blocking on newly introduced, unwaived structural regressions rather than on any pre-existing finding  
220 (Section 6.6) — because real architectures legitimately contain *intentional*, risk-accepted SPOFs that an  
221 absolute gate would flag on every build.

## 222 2.4. Structural Criticality Analysis

223 Network science offers a mature toolkit for identifying important nodes and edges. Degree, closeness  
224 and betweenness centrality, articulation points, and PageRank-style scores are prized for their efficiency and



interpretability [14, 15, 16, 17], and studies of node removal [18], cascading failure [19], and interdependent networks [20] have deepened our understanding of systemic fragility. Applied to software dependency graphs, these metrics can flag bottlenecks and single points of failure at design time.

Their limitation, for our purpose, is dimensional collapse. A single centrality score conflates mechanisms that call for different remedies: a structural single point of failure, a high-reach cascade hub, and a tightly coupled maintainability bottleneck can all present as “central,” yet a replica, a rerouting, and a decoupling refactor are not interchangeable fixes. A second limitation is representational rather than dimensional: once node and edge types are discarded, a shared library’s *simultaneous* failure mode — every consumer failing in one event rather than along a propagation path — is indistinguishable from an ordinary edge, so an untyped model cannot express it even in principle. Whether that mechanism produces a large scoring gap in practice is a separate, empirical question, which we test directly and answer in the negative for our suite (Section 5.4). Our RMAV attribution retains the interpretability that makes structural metrics attractive while decomposing criticality into orthogonal dimensions, and our typed model keeps the semantics that single-score centrality erases.

### 2.5. Learning-Based Criticality Prediction

A growing body of work learns to identify critical nodes directly from graph structure, often surpassing hand-crafted metrics when higher-order structure matters: FINDER locates key entities in networked systems, DrBC learns to approximate betweenness, and PowerGraph provides a GNN benchmark for cascading-failure and critical-node analysis in power-grid networks [21, 22, 23].

Most such methods build on the homogeneous message-passing lineage of GCN [24], GraphSAGE [25] and GAT [26], and therefore target *homogeneous* graphs. Pub-sub middleware is intrinsically heterogeneous — applications publish and subscribe to topics, topics are routed through brokers, libraries introduce code dependencies, and deployment nodes impose locality — and flattening this into a homogeneous graph discards information about how failures propagate. Heterogeneous graph neural networks address this directly: RGCN applies relation-specific transformations [27], HAN uses hierarchical attention [28], HGT parameterizes attention by node and edge type [29], and MAGNN aggregates along metapaths [30]. A known hazard in dense, hub-dominated regions is over-smoothing [31, 32]. Our learned predictor adopts relation-specific message passing over the native typed architecture for exactly these reasons, but we treat it as one of two predictors rather than the sole contribution: a central question of this paper (RQ1) is *where* such learning improves on non-learning alternatives — in recovering the full ordering, in identifying the critical set, or both — since, as Section 8.1 shows, the answer differs depending on which of those is asked about.

### 2.6. Quality Attributes and Multi-Criteria Scoring

Software quality is conventionally described along attributes such as reliability, maintainability, availability, and security, formalised in the product quality model of ISO/IEC 25010:2023 [33], and a substantial literature

connects these attributes to measurable structural and code-level properties. The quality-in-use portion of the earlier ISO/IEC 25010:2011 has since been separated into a standard of its own, **ISO/IEC 25019:2023** [34]; under that model, stakeholder harm is evaluated over three macro-characteristics: *Beneficialness* (Usability: Effectiveness, Efficiency, Satisfaction), *Freedom from Risk* (Economic, Health, Life, Environmental), and *Acceptability*. The dependability vocabulary we adopt for failure, fault and impact follows the standard taxonomy [35], and the architecture-evaluation tradition we position against is that of scenario-based methods such as ATAM [36, 37]. Combining several structural properties into a single decision score is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) offers a pairwise-comparison formalism with an explicit consistency check [38]. We use that formalism to state and audit our weights, not to elicit them from raters — a distinction we make explicit in Section 4.3, because the consistency check certifies internal coherence, not the provenance of the judgements it is applied to.

What has not been done, to our knowledge, is to use a multi-criteria decomposition as the *attribution* mechanism for pre-deployment component criticality in pub-sub systems — that is, to make the per-dimension breakdown the explanation an architect acts on, with each structural metric feeding exactly one dimension so that the reason a component is critical is legible from its profile. Our RMAV scoring does precisely this, applying the pairwise formalism both within each dimension and to form the composite  $Q(v)$ , with a shrinkage parameter that blends the stated weighting toward a uniform prior. We report the sensitivity of that shrinkage rather than assume it helps: measured against simulated impact it is monotonically harmful, and equal weights outperform the calibrated vector (Section 8.3). The contribution we claim here is therefore explanatory — the per-dimension breakdown — not an accuracy gain from the weighting. This connects the interpretability tradition of structural analysis (Section 2.4) to the decision-theoretic tradition of multi-criteria scoring and ISO/IEC 25019 Quality-in-Use, and is what distinguishes attribution here from an opaque learned score.

## 2.7. Architectural Remediation and Anti-Pattern Detection

A related strand detects architectural anti-patterns and recommends refactorings — cyclic dependencies, hubs, unstable interfaces — typically from a static dependency model, and evaluates the effect of a change by re-analyzing the modified model. Catalogues of architectural bad smells [39, 40] formalise these structures at the component-and-connector level, and the microservice literature has extended them to distributed deployments, where cyclic and hub-shaped service dependencies carry the same diagnosis [41, 42]. The underlying dependency metrics — instability, afferent and efferent coupling — descend from Martin’s design-quality criteria [43], and the technical-debt framing that motivates acting on them from Cunningham [44] and the subsequent management literature [45]. Our prescriptive stage is in this spirit but differs in its acceptance test: rather than accepting an edit because it improves a static metric, we *verify* each candidate edit on a counterfactual graph using the same discrete-event simulation oracle that produces our ground-truth impact,

and accept it only if the reduction in simulated impact exceeds a multi-seed variance threshold. Generation of candidate edits remains topology-only, preserving the independence between the diagnostic and validation paths that the rest of the framework relies on.

## 2.8. Positioning

In summary, prior approaches either (i) address pub-sub dependability at the protocol or runtime level, presupposing a deployed system; (ii) offer code-level SCA that is blind to inter-component topologies; (iii) offer structural analysis that conflates failure mechanisms and cannot represent typed modes such as simultaneous shared-library failure; (iv) apply graph learning while discarding the typed semantics of pub-sub; or (v) use multi-criteria scoring for prioritization but not as an interpretable criticality *attribution* over a typed architecture graph. Software-as-a-Graph combines a typed multigraph model, multi-dimensional attribution under an audited weighting, dual interpretable and learned impact predictors, and a simulation-verified, delta-aware continuous CI/CD quality gate. The stratified correlation evaluation we report — by node type as well as pooled — is a direct consequence of taking node and edge type seriously, and is a methodological standard the untyped or single-dimensional methods reviewed above do not apply.

## 3. The Software-as-a-Graph Model

This section defines the graph model on which all subsequent analysis operates. We first give the formal object and its node and edge types (Section 3.1), then the QoS-derived edge and vertex weights that encode coupling strength (Section 3.2), then the derivation of logical dependencies from structural edges (Section 3.3), the ingestion of code-level SCA metrics (Section 3.4), and finally the two graph views and the multi-layer projections that the attribution and impact stages consume (Section 3.5). A running example threads through the section (Section 3.6).

### 3.1. Nodes, Edges, and the Formal Object

A distributed publish–subscribe system is modeled as a typed, weighted, directed multigraph

$$G = (V, E, \tau_V, \tau_E, w_E, w_V), \quad (1)$$

where the vertex set partitions into five component types,

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}}, \quad (2)$$

the type functions  $\tau_V : V \rightarrow \{\text{App}, \text{Broker}, \text{Topic}, \text{Node}, \text{Library}\}$  and  $\tau_E$  label vertices and edges, and the weight functions  $w_E : E \rightarrow [0, 1]$  and  $w_V : V \rightarrow [0, 1]$  encode QoS-derived coupling strength. The edge set

is the disjoint union of *structural* edges imported directly from the architecture description and *dependency* edges (DEPENDS\_ON) derived from them (Section 3.3).

**Node types.** Each type corresponds to a distinct architectural element with its own failure semantics:

Table 1: Node types of the SaG model. Each type carries distinct failure semantics.

| Type               | Role                                                 | Representative instances                         |
|--------------------|------------------------------------------------------|--------------------------------------------------|
| <b>Application</b> | A process that publishes and/or subscribes to topics | ROS 2 node, Kafka producer/consumer, MQTT client |
| <b>Broker</b>      | A message-routing intermediary                       | RabbitMQ, Mosquitto, DDS middleware              |
| <b>Topic</b>       | A named message channel                              | /sensor/lidar, order.events                      |
| <b>Node</b>        | A physical or virtual host                           | server, cloud VM, embedded controller            |
| <b>Library</b>     | A shared code dependency                             | sensor driver, codec, message library            |

**Structural edge types.** Six edge types are imported from the topology description and carry the direction in which messages or hosting relationships flow:

Table 2: Structural edge types, imported directly from the architecture description.

| Edge          | Direction                       | Meaning                                    |
|---------------|---------------------------------|--------------------------------------------|
| PUBLISHES_TO  | App/Library $\rightarrow$ Topic | component produces messages on the topic   |
| SUBSCRIBES_TO | App/Library $\rightarrow$ Topic | component consumes messages from the topic |
| ROUTES        | Broker $\rightarrow$ Topic      | broker routes the topic                    |

| Edge                     | Direction                     | Meaning                                   |
|--------------------------|-------------------------------|-------------------------------------------|
| <code>RUNS_ON</code>     | App/Broker $\rightarrow$ Node | component is hosted on the node           |
| <code>CONNECTS_TO</code> | Node $\rightarrow$ Node       | direct network link between hosts         |
| <code>USES</code>        | App $\rightarrow$ Library     | application depends on the shared library |

Retaining these types — rather than collapsing them into a single “communicates-with” relation — is what later lets the framework distinguish failure mechanisms that an untyped graph cannot (Section 3.3, Section 5).

### 3.2. QoS-Aware Edge and Vertex Weights

Not all dependencies are equally consequential: a `RELIABLE/PERSISTENT` channel carrying critical data couples its endpoints far more tightly than a `BEST_EFFORT/VOLATILE` one. Edge weights encode this from the Quality-of-Service policy of each pub-sub relationship, via a two-stage computation:

$$\text{QoS\_score} = 0.30 r + 0.40 d + 0.30 p, \quad (3)$$

$$\text{size\_norm} = \min\left(\frac{\log_2(1 + \text{size\_kb})}{50}, 1.0\right), \quad (4)$$

$$w(e) = \beta \cdot \text{QoS\_score} + (1 - \beta) \cdot \text{size\_norm}, \quad \beta = 0.85, \quad (5)$$

where  $r, d, p$  are the reliability, durability, and transport-priority scores of the mediating topic, mapped from symbolic QoS values:

Table 3: QoS symbolic-value to numeric-score mapping used by the edge weight of Section 3.2.

| Dimension       | Symbolic value $\rightarrow$ score                                                                                                                                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reliability $r$ | <code>RELIABLE</code> $\rightarrow$ 1.0; <code>BEST_EFFORT</code> $\rightarrow$ 0.0                                                                                             |
| Durability $d$  | <code>PERSISTENT</code> $\rightarrow$ 1.0; <code>TRANSIENT</code> $\rightarrow$ 0.6; <code>TRANSIENT_LOCAL</code> $\rightarrow$ 0.5;<br><code>VOLATILE</code> $\rightarrow$ 0.0 |
| Priority $p$    | <code>URGENT/CRITICAL/HIGHEST</code> $\rightarrow$ 1.0; <code>HIGH</code> $\rightarrow$ 0.66; <code>MEDIUM</code> $\rightarrow$ 0.33; <code>LOW</code> $\rightarrow$ 0.0        |

The intra-QoS sub-weights are stated judgements checked for AHP consistency (Section 4.3): durability (0.40) outweighs reliability and priority (0.30 each) because durability governs message-state survival — the

precondition for resilience — whereas reliability and priority govern transient delivery quality. A floor of  $w(e) = 0.01$  keeps even zero-QoS components visible to attribution.

**Vertex weights** propagate QoS upward from incident edges, with type-specific aggregation that reflects how each component type concentrates risk:

Table 4: Type-specific vertex weight aggregation rules.

| Type        | $w_V$                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------|
| Application | $0.80 \cdot \max(w_{\text{topic}}) + 0.20 \cdot \text{mean}(w_{\text{topic}})$                |
| Broker      | $0.70 \cdot \max(w_{\text{topic}}) + 0.30 \cdot \text{mean}(w_{\text{topic}})$                |
| Node        | $\max(w)$ over all hosted applications and brokers                                            |
| Library     | $\min(1.0, w_{\text{base}} \cdot (1 + \gamma \log_2(1 + \text{DG\_in})))$ (fan-out amplified) |

The library rule is deliberately fan-out amplified: a library’s risk grows with the number of applications that depend on it, anticipating the blast-radius mechanism of Section 3.3 and Section 5.

### 3.3. Derived Dependencies: the *DEPENDS\_ON* Projection

Structural edges record physical relationships but not *logical* dependency. A subscriber and a publisher on the same topic have no direct structural edge, yet the subscriber wholly depends on the publisher for data. We therefore derive a single semantic relation, *DEPENDS\_ON*, always directed from *dependent* to *dependency* (“if the target fails, the source is affected”), through six rules:

Table 5: The six *DEPENDS\_ON* projection rules deriving logical dependencies from structural edges.

| Rule | dependency_type | Pattern (dependent $\rightarrow$ dependency)                                                        | Weight                                 |
|------|-----------------|-----------------------------------------------------------------------------------------------------|----------------------------------------|
| 1    | app_to_app      | subscriber $\rightarrow$ publisher via a shared topic<br>(incl. transitive <i>USES*1..3</i> chains) | $\max_t w(t)$<br>over shared<br>topics |
| 2    | app_to_broker   | publisher/subscriber $\rightarrow$ broker routing its topics                                        | $\max_t w(t)$<br>over routed<br>topics |
| 3    | node_to_node    | host $\rightarrow$ host, lifted from Rules 1–2 for colocated apps                                   | lifted<br>$\max w$                     |

| Rule | dependency_type  | Pattern (dependent → dependency)                                             | Weight             |
|------|------------------|------------------------------------------------------------------------------|--------------------|
| 4    | node_to_broker   | host → broker, lifted from Rule 2                                            | lifted<br>$\max w$ |
| 5    | app_to_lib       | application → library it <b>USES</b> — <b>shared-library</b><br><b>blast</b> | $w_V(\text{app})$  |
| 6    | broker_to_broker | bidirectional, two brokers sharing a host —<br><b>colocation</b>             | $w_V(\text{node})$ |

When two applications communicate over several shared topics, a single **DEPENDS\_ON** edge records the worst-case weight together with a separate coupling count:

$$\text{edge.weight} = \max_{t \in \text{shared}} w(t), \quad \text{edge.path\_count} = |\text{shared}|. \quad (6)$$

**path\_count** is kept out of the weight to preserve the  $w \in [0, 1]$  contract; a **path\_count** of 3 denotes three simultaneous failure vectors between the same pair, which is structurally more fragile than three independent single-topic links.

**Two qualitatively different failure modes.** This is the crux of the model. Rule 1 encodes *sequential cascade*: a publisher’s failure starves its subscribers, whose failure may in turn affect their dependents, propagating step by step through topics and brokers. Rule 5 encodes a *simultaneous blast*: when a shared library fails, every application that uses it fails at once, in a single event, not along a propagation path. An untyped graph cannot tell these apart — both look like ordinary edges — yet they demand different predictions and different remedies. Preserving the **app\_to\_lib** type (Rule 5) is what lets the framework represent this simultaneous-blast mechanism at all, just as preserving **broker\_to\_broker** (Rule 6) makes broker-colocation risk representable.

### 3.4. Ingestion of Code-Level SCA Metrics

To bridge the “Architecture-Code Gap,” SaG does not operate in isolation from source code. Instead, the framework integrates code-level quality attributes directly into the graph model. During the model-import stage, SaG queries static code analysis (SCA) APIs (e.g., SonarQube’s web API) or parses local SCA report artifacts to extract modular metrics for executable **Application** and shared **Library** components.

These metrics are stored as flat properties prefixed with **cm\_\*** on each component node: - **cm\_total\_loc**: Total lines of code as reported by static analysis, providing a scale proxy. - **cm\_avg\_wmc**: Average Weighted Methods per Class, representing cognitive complexity. - **cm\_avg\_lcom**: Lack of Cohesion of Methods (on a raw  $[0, 1]$  scale), indicating how fragmented classes are. - **cm\_avg\_cbo**: Coupling Between Objects, indicating intra-component code coupling. - **cm\_avg\_rfc**: Response for a Class, measuring the number of methods

invoked by a class. - **sqalet\_ratio**: Technical debt ratio as a percentage of estimated rewrite time. -  
**bugs**: Count of static bugs identified in code. - **vulnerabilities**: Count of code-level security issues.

These properties are normalized across the component population during structural analysis (Section 4.2) and feed the **Code Quality Penalty (CQP)**, ensuring that local code defects are mathematically combined with global structural dependencies.

### 3.5. Graph Views and Multi-Layer Projections

The construction produces **two complementary views** of the same system, and the separation between them is load-bearing for the framework’s validity:

- $G_{\text{structural}}$  — the imported structural graph, used by the discrete-event simulators to generate the ground-truth impact labels (Section 5.1).
- $G_{\text{analysis}}(\ell)$  — the layer-projected **DEPENDS\_ON** graph, on which all structural metrics, quality attribution, and prediction are computed (Section 4).

Because attribution is computed on  $G_{\text{analysis}}$  while ground truth is generated by simulating  $G_{\text{structural}}$ , the predictor’s inputs are kept disjoint from the label-producing path — the **independence guarantee** that makes the pre-deployment claims of Section 4–Section 5 non-circular. We state and rely on this property throughout.

$G_{\text{analysis}}$  is filtered into four analytical layers, each isolating a component scope, a dependency subset, and the quality dimension it most informs:

Table 6: The four analytical layer projections and the quality dimension each most informs.

| Layer          | Projection            | Vertices                     | Dependency types                                                             | Quality focus   |
|----------------|-----------------------|------------------------------|------------------------------------------------------------------------------|-----------------|
| Application    | $\pi_{\text{app}}$    | App, Library                 | <b>app_to_app</b> , <b>app_to_lib</b>                                        | Reliability     |
| Infrastructure | $\pi_{\text{infra}}$  | Node                         | <b>node_to_node</b>                                                          | Availability    |
| Middleware     | $\pi_{\text{mw}}$     | Broker (in App/Node context) | <b>app_to_broker</b> ,<br><b>node_to_broker</b> ,<br><b>broker_to_broker</b> | Maintainability |
| System         | $\pi_{\text{system}}$ | all five types               | all six                                                                      | Overall         |

The middleware layer includes Application and Node vertices in the subgraph to preserve incoming edges, but reports results only for Brokers. Components further aggregate along a MIL-STD-498 [46] hierarchy —  $\text{CSU} \rightarrow \text{CSC} \rightarrow \text{CSCI} \rightarrow \text{CSS}$  — so that criticality can be rolled up from a unit to a configuration item



to the whole system, supporting reporting at whatever granularity an organization’s software configuration management already uses.

### 3.6. Running Example

Consider three applications  $a_1, a_2, a_3$ , where  $a_1$  publishes to a topic  $t$  that  $a_2$  and  $a_3$  subscribe to, all three depending on a shared library  $\ell$ . A single broker  $b$  routes  $t$ , and one host  $n$  runs all four processes. The topic declares **RELIABLE/TRANSIENT\_LOCAL/HIGH** with a 1 KiB payload, which by Section 3.2 gives it a weight of  $w(t) = 0.596$ . The structural graph records  $a_1 \xrightarrow{\text{pub}} t$ ,  $a_2, a_3 \xrightarrow{\text{sub}} t$ ,  $b \xrightarrow{\text{routes}} t$ ,  $a_i, b \xrightarrow{\text{runs\_on}} n$ , and  $a_i \xrightarrow{\text{uses}} \ell$ . Derivation adds  $a_2 \rightarrow a_1$  and  $a_3 \rightarrow a_1$  (**app\_to\_app**, Rule 1),  $a_i \rightarrow b$  (**app\_to\_broker**, Rule 2),  $n \rightarrow b$  (**node\_to\_broker**, Rule 4) and  $a_i \rightarrow \ell$  (**app\_to\_lib**, Rule 5). The two structures encode different risks: losing  $a_1$  degrades  $a_2$  and  $a_3$  through a cascade that the simulator propagates over time, whereas losing  $\ell$  fails  $a_1, a_2, a_3$  simultaneously. A topology-only centrality score ranks  $\ell$  by ordinary connectivity and cannot represent that its single failure collapses the whole component group at once. Whether that representational difference translates into a scoring gap large enough to matter is an empirical question we test in Section 5.4 (on our synthetic suite, it does not).

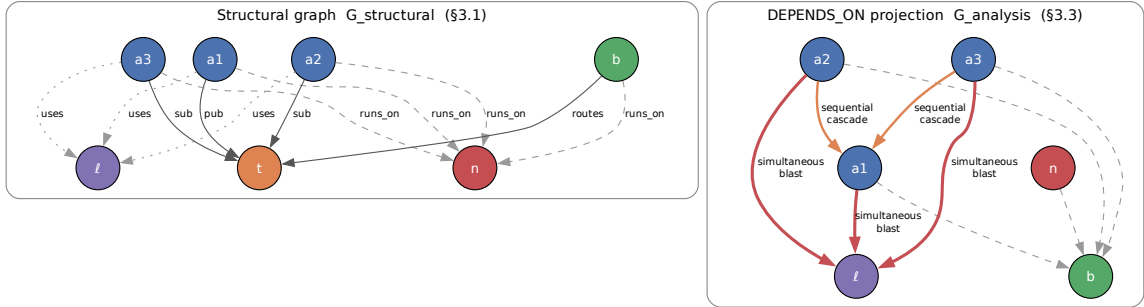


Figure 2: the running example’s structural graph and its derived **DEPENDS\_ON** projection, with sequential-cascade and simultaneous-blast edges visually distinguished.

## 4. Multi-Dimensional Quality Attribution (The Interpretable Path)

Centrality answers *whether* a component is important with a single number. An architect choosing between a replica, a reroute, and a decoupling refactor needs to know *why*. This section presents the framework’s primary diagnostic: a decomposition of each component’s criticality into four orthogonal quality dimensions, each computed from disjoint structural metrics, and combined into an interpretable composite

score. Because the dimensions do not share inputs, a component’s profile is itself the explanation of its risk — and the explanation maps directly to a remedy (Section 6).

#### 4.1. Four Orthogonal Dimensions and Formal Definitions

We attribute criticality along Reliability, Maintainability, Availability, and Vulnerability (RMAV). Grounded in **ISO/IEC 25019:2023 (Quality-in-Use)**, criticality represents the counterfactual loss of beneficialness, freedom from risk, and acceptability experienced by stakeholders if an architectural element fails. Each dimension speaks to a formal stakeholder class:

Table 7: The four RMAV dimensions, the architectural question each answers, and the stakeholder and engineering role each routes to.

| Dim.     | Architectural Question                         | High score means                            | Harmed Stakeholder (ISO 25019)                                                                                                   | Secondary Stakeholder (Engineering Role) |
|----------|------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>R</b> | How broadly and deeply does failure propagate? | Failure cascades widely; hard to contain    | <b>Primary &amp; Indirect:</b> operators and downstream beneficiaries whose tasks retry, fail over, or degrade                   | Reliability Engineer                     |
| <b>M</b> | How hard is this to change safely?             | Tightly coupled structural bottleneck       | <b>Secondary:</b> maintainers facing high regression likelihood upon refactoring                                                 | Software Architect                       |
| <b>A</b> | Is this a structural single point of failure?  | Removing it partitions the dependency graph | <b>Primary &amp; Indirect:</b> direct operators (traders, clinicians, drivers) and dependent beneficiaries facing task cessation | DevOps / SRE                             |

| Dim. | Architectural<br>Question                         | High score<br>means                                                                                               | Harmed Stakeholder (ISO<br>25019)                                                                                      | Secondary Stakeholder<br>(Engineering Role) |
|------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| V    | How attractive<br>a target is this<br>for attack? | Central and<br>reachable on<br>$G^\top$ , with<br>many<br>strongly-<br>guaranteed<br>flows<br>converging<br>on it | <b>Primary, Indirect &amp;<br/>business:</b> parties relying on<br>the guarantees an attacker<br>would gain control of | Security Engineer                           |

Maintainability is the one dimension whose direct victim is the secondary stakeholder; the other three route a finding to the engineering role equipped to act on it while denominating severity in harm to primary and indirect stakeholders. The  $V$  row is deliberately phrased in terms of *guarantees* rather than asset value:  $w_{\text{in}}$  is a delivery-guarantee proxy, and the model carries no field for data sensitivity, privilege, or PII (Section 9.2, Section 9.3).

Four formal definitions establish the theoretical construct. Each is stated in full, because several clauses that are easy to skim past do real work in what follows.

**Definition D1 — Component Criticality.** The degree to which the failure, latency, or functional degradation of a specific software component — directly or transitively — reduces the system’s capacity to enable its stakeholders to achieve specified operational goals with beneficialness (usability, accessibility, suitability), freedom from risk (economic, health, life, environmental), and acceptability (experience, trustworthiness, compliance) within its operational context. Realised at layer  $l$  as a measure  $\text{crit}_l : V_l \rightarrow [0, 1]^4 \times [0, 1]$  mapping each  $v \in V_l$  to  $\mathbf{s}(v) = [R(v), M(v), A(v), V(v)]^T$  and composite  $Q(v)$ .

“*Failure, latency, or functional degradation*” names three distinct fault modes. The structural estimator does not separate them — RMAV scores a component’s *exposure*, which is why one score covers all three — whereas the simulation oracle does (Section 5.1). “*Directly or transitively*” is why Reliability exists as a dimension separate from Availability: the harm is loss of stakeholder outcomes reachable *through* the component, not loss of graph connectivity. “*Within its operational context*” is the clause Section 4.3 operationalises through the QoS-profile adaptation of the composite weights.

**Definition D2 — Relationship Criticality.** The degree to which the disruption, latency,

or data loss across a specific inter-component interaction or dependency path — **with both endpoint components remaining operational** — reduces the system’s capacity to enable its stakeholders to achieve specified goals with beneficialness, freedom from risk, and acceptability, **in proportion to the absence of redundant or fallback paths around it**. Realised at layer  $l$  as  $\text{crit}_l : E_l \rightarrow [0, 1]^4 \times [0, 1]$ , the same signature as D1.

The first emphasised clause is what makes D2 more than D1 restated for edges: it isolates the *partial-outage* case, in which the component is up and its dashboards are green while one data flow has stopped. It is also exactly the condition the edge oracle enforces (Section 8.2). The second clause makes replaceability *scale* the harm rather than gate it, which is why only the Availability dimension is bridge-gated while R, M and V score replaceable links too (Section 4.7).

**Definition D3 — Criticality is a consequence, not a risk.** Under the standard decomposition of risk into likelihood and consequence, criticality as defined here is the **consequence factor alone**. No RMAV dimension estimates how probable it is that a component or relationship fails; every dimension estimates how much is lost *given* that it does.

Two consequences bear directly on how the results of Section 8 should be read. Ranking  $u$  above  $v$  says that losing  $u$  hurts more, not that  $u$  is more likely to be lost — so every comparison in this paper holds likelihood fixed. And restricting the construct to consequence is precisely what makes it computable pre-deployment: consequence follows from architecture, which exists before the system runs; likelihood follows from behaviour, which does not.

**Definition D4 — Criticality is relative, not absolute.** Every score and tier is relative to (i) the score distribution of the system  $S$  being analysed, since tiers are box-plot thresholds over that distribution (Section 4.4), and (ii) the layer  $l$ , since both the vertex set being ranked and the weight normalisation change with the projection. Criticality values are therefore **not comparable across systems or across layers**.

A well-designed redundant system still has a CRITICAL tier, and a system full of SPOFs still has a MINIMAL tier; the tier prioritises attention inside one system rather than comparing two. D4 also constrains how this paper may aggregate: any figure computed over more than one scenario must be formed from within-scenario ranks or per-scenario statistics, never from raw scores pooled across systems. Section 5.5 and Section 8.5 carry the corresponding scoping statements.

For **components**, the dimensions are **orthogonal by construction**: each raw structural metric feeds exactly one dimension, never more. This is a deliberate design constraint, not an empirical observation — allowing a metric into two dimensions would silently inflate its weight relative to the stated weighting

(Section 4.3). Orthogonality is what makes the breakdown legible: a pure single point of failure scores high on A but low on R, M, and V; a god-component scores high on M; a cascade hub scores high on R. The *shape* of the profile names the failure mode. The constraint is specific to the component decomposition; the edge formulas of Section 4.7 deliberately relax it in exchange for endpoint context, and we say so there rather than letting the claim read as framework-wide.

#### 4.2. RMAV Formulas

All metric inputs are rank-normalized to  $[0, 1]$ , so every RMAV score lies in  $[0, 1]$ . Table 8 fixes notation for every structural metric the four formulas below consume; each is computed once on  $G_{\text{analysis}}$  and feeds exactly one RMAV dimension (Section 4.1).

Table 8: RMAV input metric notation.  $G^\top$  denotes the transpose of the DEPENDS\_ON graph (the failure-propagation direction, since edges point dependent  $\rightarrow$  dependency).

| Symbol                  | Name                             | Computed as                                                                     | Feeds                     |
|-------------------------|----------------------------------|---------------------------------------------------------------------------------|---------------------------|
| RPR( $v$ )              | Reverse PageRank                 | PageRank on $G^\top$ ( $d = 0.85$ )                                             | $R$                       |
| DG_in( $v$ )            | In-degree<br>(rank-norm.)        | Direct dependent count on DEPENDS_ON                                            | $R$                       |
| MPCI( $v$ )             | Multi-Path Coupling Index        | $\sum_{e \in \text{InEdges}(v)} \max(\text{path\_count}(e) - 1, 0) / ( V  - 1)$ | $R$ (via CDPot_enh)       |
| CDPot_enh( $v$ )        | Enhanced Cascade Depth Potential | RPR/DG_in blend, amplified by MPCPI (Eq. above)                                 | $R$                       |
| FOC( $v$ )              | Fan-Out Criticality              | frequency- and QoS-weighted subscriber fan-out (Topic nodes only)               | $R_{\text{topic}}$        |
| BT( $v$ )               | Betweenness centrality           | Brandes' algorithm on $G_{\text{analysis}}$ , QoS-inverted edge distances       | $M$                       |
| $w_{\text{out}}(v)$     | QoS-weighted out-degree          | $\sum_{(v,u)} w(v,u)$ over outgoing dependencies                                | $M$                       |
| CQP( $v$ )              | Code Quality Penalty             | SonarQube-derived composite (Section 3.4); 0 for non-App/Library types          | $M$                       |
| CouplingRisk_enh( $v$ ) | Enhanced coupling risk           | in/out-degree balance amplified by path complexity                              | $M$                       |
| CC( $v$ )               | Clustering coefficient           | Watts–Strogatz local clustering on the undirected projection                    | $M$ (as $1 - \text{CC}$ ) |
| AP_c_directed( $v$ )    | Directed articulation score      | max of directed in/out articulation scores                                      | $A$                       |
| QSPOF( $v$ )            | QoS-weighted SPOF severity       | $\text{AP\_c\_directed}(v) \cdot w(v)$                                          | $A$                       |
| BR( $v$ )               | Bridge ratio                     | fraction of $v$ 's undirected edges that are bridges                            | $A$                       |

| Symbol     | Name                | Computed as                                      | Feeds |
|------------|---------------------|--------------------------------------------------|-------|
| CDI( $v$ ) | Connectivity        | normalized increase in average path length       | $A$   |
|            | Degradation Index   | when $v$ is removed                              |       |
| REV( $v$ ) | Reverse eigenvector | eigenvector centrality on $G^\top$               | $V$   |
|            | centrality          |                                                  |       |
| RCL( $v$ ) | Reverse closeness   | harmonic centrality on $G^\top$ , normalized by  | $V$   |
|            | (harmonic)          | $ V  - 1$                                        |       |
| $w\_in(v)$ | QoS-weighted        | $\sum_{(u,v)} w(u,v)$ over incoming dependencies | $V$   |
|            | in-degree (QADS)    |                                                  |       |

**Reliability** — fault-propagation risk. Because `DEPENDS_ON` points *dependent*  $\rightarrow$  *dependency*, a failure propagates *against* edge direction; RPR (computed on the transpose  $G^\top$ ) therefore traverses the natural failure-propagation path. For Topic nodes, which have no `DEPENDS_ON` in-degree, a fan-out form is dispatched by  $\tau_V(v)$ :

$$R(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{DG\_in}(v) + 0.25 \cdot \text{CDPot\_enh}(v) \quad [\tau_V(v) \neq \text{Topic}] \quad (7)$$

$$\text{CDPot\_enh}(v) = \min\left(\frac{\text{RPR}(v) + \text{DG\_in}(v)}{2} \cdot (1 - \min(\frac{\text{out\_degree\_raw}(v)}{\max(\text{in\_degree\_raw}(v), \epsilon)}, 1)) \cdot (1 + \text{MPCI}(v)), 1.0\right) \quad (8)$$

$$R_{\text{topic}}(v) = 0.50 \cdot \text{FOC}(v) + 0.50 \cdot \text{CDPot\_topic}(v), \quad \text{CDPot\_topic}(v) = \text{FOC}(v) (1 - \min(\text{publisher\_count\_norm}(v), 1)) \quad (9)$$

**Maintainability** — coupling complexity:

$$M(v) = 0.35 \text{BT}(v) + 0.30 w\_out(v) + 0.15 \text{CQP}(v) + 0.12 \text{CouplingRisk\_enh}(v) + 0.08 (1 - \text{CC}(v)), \quad (10)$$

$$\text{CQP}(v) = 0.10 \text{loc\_norm} + 0.35 \text{complexity\_norm} + 0.30 \text{instability\_code} + 0.25 \text{lcom\_norm}. \quad (11)$$

Here, the Code Quality Penalty (CQP) translates local code-level fragility into system-level maintainability risk. The components `loc_norm`, `complexity_norm`, and `lcom_norm` represent the min-max normalized values of the ingested SonarQube properties `loc`, `cyclomatic_complexity`, and `lcom`, respectively. These are calculated independently for Applications and Libraries to prevent scale differences from distorting the normalization. The metric `instability_code` represents class instability (efferent coupling divided by total coupling). The CQP thus ensures that local code debt is penalised, but only as a sub-factor of Maintainability ( $M$ ), which remains heavily weighted by topological metrics such as betweenness centrality ( $BT$ ) and efferent QoS-weighted out-degree ( $w\_out$ ). CQP is zero for non-Application/Library types (graceful degradation). The two instability signals are intentional and distinct: `instability_code` is static-code fragility (local); `CouplingRisk_enh` is runtime-topology fragility (global).

**Availability** — single-point-of-failure risk:

$$A(v) = 0.35 \text{ AP\_c\_directed}(v) + 0.25 \text{ QSPOF}(v) + 0.25 \text{ BR}(v) + 0.10 \text{ CDI}(v) + 0.05 w(v). \quad (12)$$

The directed articulation score (rather than the undirected AP, which both over- and under-reports in pub-sub graphs) captures directed cut vertices; QSPOF amplifies it by the component’s QoS weight, so a SPOF carrying critical traffic is scored as doubly severe.

**Vulnerability** — adversarial exposure:

$$V(v) = 0.40 \text{ REV}(v) + 0.35 \text{ RCL}(v) + 0.25 w_{\text{in}}(v). \quad (13)$$

All three terms are computed on the transpose to model attack propagation and adversarial reach toward high-SLA surfaces.

#### 4.3. The Composite Score $Q(v)$

The four dimensions combine into a composite criticality score under a stated weighting:

$$Q(v) = w_A A(v) + w_R R(v) + w_M M(v) + w_V V(v). \quad (14)$$

**The weights are stated design judgements, audited for coherence rather than elicited.** Each comparison matrix is written on Saaty’s 1–9 scale to express an intended ordering, then checked with the Analytic Hierarchy Process [38]: row geometric means normalised to a weight vector, with a consistency ratio  $\text{CR} = \text{CI}/\text{RI}$  required to satisfy  $\text{CR} \leq 0.10$ . We describe them as “stated and audited” rather than “AHP-derived” because the resulting near-zero consistency ratios ( $\text{CR} < 0.02$ , and below 0.002 on the  $5 \times 5$  intra-dimension matrices) are a symptom of the construction: a matrix filled in from a target weight vector is consistent almost by construction, whereas genuine multi-rater elicitation on five criteria rarely lands that low. The audit certifies internal coherence, not provenance.

**Three weighting paths exist in the implementation, and the reported results use the first.**

We set them out explicitly, because they do not coincide and an earlier version of this paper conflated them:

Table 9: The three weighting paths in the implementation. Reported results use the stated default.

| Path                  | Composite<br>( $w_A, w_R, w_M, w_V$ ) | Intra-dimension                                    | Used by                                                  |
|-----------------------|---------------------------------------|----------------------------------------------------|----------------------------------------------------------|
| <b>Stated default</b> | (0.43, 0.24, 0.17, 0.16)              | exactly the coefficients<br>printed in Section 4.2 | <b>all reported results</b><br>(Section 8.1–Section 8.5) |

| Path                                 | Composite<br>( $w_A, w_R, w_M, w_V$ )                                                                                        | Intra-dimension                          | Used by                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------|
| AHP reconstruction,<br>$\lambda = 1$ | (0.458, 0.246, 0.169, 0.128)                                                                                                 | matches Section 4.2 to<br>three decimals | upper endpoint of the<br>Section 8.3 sweep |
| AHP with shrinkage $\lambda$         | $\lambda w_{\text{AHP}} + (1 - \lambda) \frac{1}{n_{\text{dim}}}$ ; at<br>$\lambda = 0.70$ ,<br>(0.395, 0.247, 0.193, 0.165) | shrunk likewise                          | the Section 8.3<br>sensitivity sweep       |

All three place Availability first (a SPOF is a certain graph partition), Reliability second (cascade reach), then Maintainability and Vulnerability. Shrinkage blends toward a uniform prior and is applied to the intra-dimension vectors as well as to the composite, so  $\lambda$  moves every RMAV formula at once; it exists because weight vectors from small comparison sets can be extreme. The stated default is *not* a point on that  $\lambda$  axis — it is a hand-set vector expressing the same ordering — which is why Section 8.3 reports the sweep as a sensitivity analysis of the ordering rather than as a tuning curve for a deployed parameter.

**A QoS-profile adaptation is applied on top of whichever vector is in force.** Before scoring, the four composite coefficients are re-derived from the analysed system’s aggregate QoS profile and renormalised to sum to one: a predominantly PERSISTENT/RELIABLE/high-priority system shifts weight toward  $R$  and  $A$ , a predominantly VOLATILE/BEST\_EFFORT one toward  $M$  and  $V$ , and a mixed profile keeps the stated defaults. This is D1’s “*within its operational context*” clause made computable, and it is on by default in every run reported here. Two consequences follow. The effective composite is therefore **per system**, so the vectors tabulated above are starting points rather than the coefficients any individual system is scored with — a further sense in which D4’s relativity holds. And it does not disturb the determinism of Section 4.5: the adaptation is a deterministic function of the same  $G_{\text{analysis}}$ , with no learned or stochastic component.

**Quality-in-Use Transformation Matrix.** To connect product-quality mechanisms ( $R, M, A, V$ ) to ISO/IEC 25019 Quality-in-Use harms, the vector  $\mathbf{s}_{\text{RMAV}}(v) = [R(v), M(v), A(v), V(v)]^T$  projects into stakeholder harm scores  $[H_{\text{Ben}}, H_{\text{Risk}}, H_{\text{Acc}}]^T$  via transformation matrix  $\mathbf{M}_{\text{RMAV} \rightarrow \text{QiU}}$ :

$$\mathbf{h}_{\text{QiU}}(v) = \mathbf{M}_{\text{RMAV} \rightarrow \text{QiU}} \cdot \mathbf{s}_{\text{RMAV}}(v) = \begin{bmatrix} 0.35 & 0.25 & 0.40 & 0.00 \\ 0.10 & 0.00 & 0.50 & 0.40 \\ 0.30 & 0.00 & 0.20 & 0.50 \end{bmatrix} \begin{bmatrix} R(v) \\ M(v) \\ A(v) \\ V(v) \end{bmatrix}. \quad (15)$$

In a specific deployment domain, Quality-in-Use loss can be further parametrized by a **Domain Context Vector**  $\vec{\omega}_{\text{domain}} = [\omega_{\text{Ben}}, \omega_{\text{Risk}}, \omega_{\text{Acc}}]$  that reweights the three harm scores — safety-critical ROS 2 prioritising Freedom from Risk, financial HFT prioritising Efficiency under Beneficialness, and so on.

**Both  $\mathbf{M}_{\text{RMAV} \rightarrow \text{QiU}}$  and  $\vec{\omega}_{\text{domain}}$  are stated mappings, and neither is used in any result reported**



in this paper. They are given here because D1 and D2 define criticality on Quality-in-Use while the four dimensions are named after product quality, and a reader is owed an explicit statement of how one is meant to reach the other. But the coefficients are asserted, not fitted or elicited; unlike the composite weights they carry no consistency audit; and no table in Section 8 reports an  $\mathbf{h}_{\text{QIU}}$  score. They should be read as a specification of the intended correspondence, not as a validated instrument. Deriving them — and testing whether per-domain reweighting recovers the ranking accuracy that the global weighting does not — is future work (Section 9.3).

**We report the sensitivity of the composite weighting, and it is not favourable.** Sweeping  $\lambda$  over  $\{0, \dots, 1\}$  against simulated impact shows no plateau at any value and a monotone decline in  $\rho$ , with equal weights ( $\lambda = 0$ ) outperforming the  $\lambda = 0.70$  setting by 0.111 (Section 8.3). An earlier version of this paper reported a plateau over  $\lambda \in [0.65, 0.75]$ ; that claim was not supported by a committed artifact and does not survive measurement. Because the decline is monotone across the whole range, the conclusion applies to the stated default of the table above as well, even though that vector is not itself a point on the  $\lambda$  axis: every weighting that expresses the intended ordering is beaten by the uniform one on this cohort.

One reading of the decline is that a single global weighting cannot fit scenarios drawn from domains whose harm profiles genuinely differ, which is what  $\vec{\omega}_{\text{domain}}$  is meant to express. We flag that as a conjecture rather than an explanation: we have not run the per-domain reweighting that would test it, and until we do, the measured fact is simply that the stated weighting does not improve ranking. We keep the decomposition and drop the accuracy claim attached to its weighting. The four dimensions earn their place by being *separately actionable* — a structural single point of failure and a cascade hub have different owners and different remedies even at identical composite scores (Section 4.1) — and that property is independent of how the four are combined into a scalar. A practitioner optimising purely for ranking should use equal weights; a practitioner who needs to know *why* a component is critical needs the profile, whatever the weights.

#### 4.4. Adaptive Criticality Classification

A raw  $Q(v)$  is most useful when turned into an action threshold relative to the system’s own distribution rather than an absolute cutoff. We classify with an adaptive box-plot rule, applied independently to each RMAV dimension and to the composite:

$$\text{CRITICAL} : Q > Q_3 + 1.5 \text{ IQR}; \quad \text{HIGH} : Q_3 < Q \leq \text{upper fence}; \quad \text{MEDIUM} : \text{med} < Q \leq Q_3; \quad (16)$$

$$\text{LOW} : Q_1 < Q \leq \text{med}; \quad \text{MINIMAL} : Q \leq Q_1. \quad (17)$$

Per-dimension classification is what makes the output actionable: a component can be CRITICAL on Availability yet MINIMAL on Vulnerability, which tells the architect to add a replica rather than to harden an interface. For small graphs ( $n < 12$ ), where quartile fences are unstable, a percentile fallback is used

(CRITICAL = top 10%, HIGH = 75th–90th, MEDIUM = 50th–75th, LOW = 25th–50th, MINIMAL = bottom 25%).

#### 4.5. Determinism and the Independence Guarantee

Attribution is fully deterministic and interpretable: the same  $G_{\text{analysis}}$  always yields the same scores, with no learned parameters and no stochastic component. Critically, every input to  $Q(v)$  is a structural metric of  $G_{\text{analysis}}$ ; none derives from the discrete-event simulation that produces the ground-truth impact labels used to evaluate the framework (Section 5.1, Section 7.5). This is the **independence guarantee**: the attribution path and the label path are disjoint, so a correlation between  $Q(v)$  and simulated impact — under either oracle — measures genuine predictive content rather than information leaked from the labels into the score.

#### 4.6. Worked Attribution

Scoring the running example of Section 3.6 with the pipeline of Section 4.2–Section 4.4 gives the following profile. The point of the table is the divergence between the last two columns:

Table 10: Worked RMAV attribution for the running example of Section 3.6. The divergence between the last two columns is the point.

| Component         | $R$   | $M$   | $A$   | $V$   | $Q$   | Composite tier | Dominant dimension tier           |
|-------------------|-------|-------|-------|-------|-------|----------------|-----------------------------------|
| $b$ (broker)      | 0.569 | 0.278 | 0.335 | 0.375 | 0.356 | LOW            | <b>CRITICAL on <math>A</math></b> |
| $t$ (topic)       | 0.875 | 0.305 | 0.021 | 0.188 | 0.260 | MINIMAL        | <b>CRITICAL on <math>R</math></b> |
| $a_1$ (publisher) | 0.500 | 0.627 | 0.021 | 0.667 | 0.428 | MEDIUM         | <b>CRITICAL on <math>M</math></b> |
| $n$ (host)        | 0.300 | 0.405 | 0.271 | 0.438 | 0.357 | MEDIUM         | HIGH on $A$                       |
| $\ell$ (library)  | 0.450 | 0.357 | 0.050 | 0.333 | 0.266 | LOW            | LOW on $R, M, V$                  |

Three components illustrate how the profile names the failure mode, and each is a case the composite alone would mislead on. The broker  $b$  is a directed cut vertex: removing it partitions the graph, so it is CRITICAL on  $A$  — driven by the directed articulation score and, because  $t$  carries RELIABLE/TRANSIENT\_LOCAL/HIGH traffic at  $w(t) = 0.596$ , by QSPOF — while scoring MINIMAL on  $M$ . Yet its *composite* tier is LOW. An architect reading only  $Q(b)$  would deprioritise the one component whose loss stops every dependent outright; the  $A$  tier is what routes it to the SRE for a second broker. The topic  $t$  inverts the same pattern on a different dimension: CRITICAL on  $R$  through its subscriber fan-out, MINIMAL overall. The publisher  $a_1$  is CRITICAL on  $M$  — a betweenness and efferent-coupling bottleneck the architect should decouple — at a composite of only MEDIUM.

This is the concrete form of the claim Section 8.3 arrives at empirically. The composite is a ranking device and, on this cohort, not a good one; the *profile* is the diagnostic, and the two disagree often enough that reading only the scalar discards the finding. Reading the broker row as a stakeholder statement, in the terms of Section 4.1: if  $b$  fails,  $a_2$  and  $a_3$  lose their only path to  $t$ , so the monitoring task does not degrade — it stops. That is a Beneficialness/Effectiveness loss, and the outage window is itself a Freedom-from-risk exposure. What the score does *not* say is how often  $b$  fails or how fast it would be restored (D3); a CRITICAL tier is a statement about structural exposure to Quality-in-Use loss, not a measurement of Quality-in-Use loss itself (Section 9.2).

The edge scores of Section 4.7 add the complementary reading. Only one dependency in the example is a bridge —  $n \rightarrow b$ , at  $A = 0.354$  and  $Q = 0.320$ , the highest-scoring edge — while every `app_to_broker`, `app_to_app` and `app_to_lib` edge scores  $A = 0.004$ : replaceable links, whose loss costs Efficiency rather than Effectiveness. Those replaceable edges nonetheless carry  $R \approx 0.29$ , because  $w(e) = 0.596$  and their endpoints' own reliability reach them through the endpoint term. That is D2's proportionality clause behaving as specified: redundancy scales the harm to near zero on  $A$  without switching the other three dimensions off.

The shared library  $\ell$  illustrates the qualitatively distinct simultaneous-blast mechanism of Rule 5 (Section 3.3): its individual structural centrality need not be remarkable, yet its failure collapses  $a_1, a_2, a_3$  at once, in a single event rather than a propagation chain. Whether this mechanism produces a low- $Q$ /high- $I$  mismatch in practice is an empirical question we evaluate directly in Section 5.4 (on our synthetic suite, it does not); independent of that, the mechanism is why the FanOutReduction operator (Section 6) is triggered by structural blast signals rather than by  $Q(v)$  itself — a library's consumer fan-out is legible from structure alone, before any simulation is run.

#### 4.7. Relationship Criticality

D2 gives edges the same signature as nodes, and this section supplies the corresponding measure. The motivation is that an edge failure and a node failure produce different observable symptoms. A node failure is a *total* outage of a capability: everything the component provides stops. An edge failure is a *partial* outage: the component is up, its other consumers are fine, its dashboards are green — but one data flow has stopped, and for the stakeholder behind that link, Effectiveness is lost just as completely as in a full outage. Two cases follow that endpoint scores cannot express. A high-criticality node may have uniformly low-criticality edges, as with a redundantly connected broker where losing any single link changes nothing; and a low-criticality node may sit behind a single highly critical bridge edge, where losing that one relationship is as consequential for its dependents as losing a much higher-scoring component.

**Structural edge signals.** Four per-edge quantities are computed on  $G_{\text{analysis}}$ :

Table 11: Per-edge structural signals computed on  $G_{\text{analysis}}$  for relationship criticality.

| Signal                          | Computed as                                                                        | Reads as                                                                                                 |
|---------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| $\mathbf{1}_{\text{bridge}}(e)$ | cut-edge test on the undirected projection                                         | removing $e$ disconnects a subgraph — the Effectiveness case                                             |
| $\text{bt}(e)$                  | edge betweenness on <b>inverted</b> weights, each edge’s length $1/w(e)$           | fraction of shortest dependency paths crossing $e$ — the Efficiency case (how much traffic must reroute) |
| $w(e)$                          | worst-case (max) QoS weight over the topics mediating the dependency (Section 3.3) | how strongly the flow across $e$ is guaranteed                                                           |
| $\text{path\_count}(e)$         | number of distinct mediating topics or shared hosts                                | coupling intensity, kept out of $w(e)$ to preserve $w \in [0, 1]$                                        |

Weight inversion is what makes strongly-guaranteed dependencies *short*, so they attract shortest paths rather than repelling them. Unlike the node case,  $w(e)$  enters **un-normalised**: the Section 3.2 construction already places it in  $[0, 1]$ .

**Edge RMAV.** Each edge is scored on the same four dimensions, blending its intrinsic signals with the endpoint scores of Section 4.2:

$$R(u, v) = 0.35 \text{bt} + 0.30 w(e) + 0.20 \max(R(u), R(v)) \quad (18)$$

$$M(u, v) = 0.35 \text{bt} + 0.30 \mathbf{1}_{\text{bridge}} + 0.15 w(e) \quad (19)$$

$$A(u, v) = 0.30 \mathbf{1}_{\text{bridge}} + 0.20 \min(A(u), A(v)) \quad (20)$$

$$V(u, v) = 0.15 w(e) + 0.20 \max(V(u), V(v)) \quad (21)$$

combined into  $Q(u, v)$  with the same composite coefficients and QoS-profile adaptation as a node (Section 4.3), and classified by the same box-plot rule (Section 4.4) applied within the edge set.

Four design choices carry meaning. **max for  $R$  and  $V$ , min for  $A$** : a link is only as reliable or as secure as its *riskiest* endpoint, since failure or compromise on either side propagates across it, but only as available as its *weakest*, since the edge cannot be more resilient than the more fragile side it connects.  **$\mathbf{1}_{\text{bridge}}$  appears in both  $M$  and  $A$** : a non-redundant edge is expensive to route around (an Efficiency cost to the engineering stakeholder) *and* a structural cut-point if removed (an Effectiveness loss to the end user) — one structural

fact, two stakeholder consequences.  $w(e)$  **appears in  $R$ ,  $M$  and  $V$  but not  $A$** : the guarantee crossing a link scales how much its loss costs, but not whether it can be lost at all. Replaceability is topological; consequence is QoS-weighted. This is D2’s redundancy clause made operational — only  $A$  is bridge-gated, while the other three score replaceable links too. **path\_count does not enter the edge score directly**; it shapes the endpoints’  $R$  and  $M$  (Section 4.2), of which only  $R$  reaches the edge again, through the endpoint term.

**Two scoping conditions.** First, the orthogonality constraint of Section 4.1 is a property of the *node* decomposition and does not carry over here:  $bt$  feeds both  $R$  and  $M$ ,  $1_{\text{bridge}}$  feeds both  $M$  and  $A$ , and  $w(e)$  feeds three of the four. The edge formulas trade orthogonality for the endpoint context that distinguishes an edge score from a node score, and we state the claim as node-scoped rather than framework-wide. Second, the four edge dimensions do not draw on equal coefficient mass —  $R$  sums to 0.85 of a possible 1.0,  $M$  to 0.80,  $A$  to 0.50,  $V$  to 0.35 — so raw edge scores are comparable *within* a dimension but not *across* dimensions. Because classification is box-plot relative within the edge set, per-dimension rankings and tiers are unaffected; only the raw magnitudes are. An edge’s dimension *tiers* should be read, not its absolute dimension values.

**What validates this, and what does not.** Relationship attribution is scored over  $G_{\text{analysis}}$  — the derived `DEPENDS_ON` edges — while the edge-removal oracle of Section 8.2 severs raw edges of  $G_{\text{structural}}$ . On `av_system` those are 3,753 derived edges against a candidate set of 50 raw structural edges drawn predominantly from `RUNS_ON` and `CONNECTS_TO`, with a handful of `SUBSCRIBES_TO` and `PUBLISHES_TO` relations (Section 8.2 gives the exact composition), and the two populations barely intersect. This is not an oversight: it is the independence guarantee of Section 5.3 operating exactly as designed — predictors and labels must be computed over disjoint graph views — and the edge case simply has no shared identifier space for the two views to meet on, where the node case does. **There is therefore no common edge population on which  $Q(u, v)$  and the measured edge impact are both defined**, and the correlation-style validation applied to node scores in Section 8.1 cannot be run for edges as the two quantities are currently constructed. We present relationship attribution as a *defined and implemented* measure that operationalises D2, and the edge-removal measurement of Section 8.2 as a separate result about the structural graph — not as a validation of the attribution, and we do not report or imply a correlation between the two anywhere in this paper. Re-simulating on `DEPENDS_ON` directly is not an available fix: the framework’s independence guarantee (Section 5.3) requires simulation to operate only on  $G_{\text{structural}}$ . The one route that would close the gap without violating that guarantee — tracking, for each derived edge, which raw structural edges mediate it, then aggregating their measured impact onto it — is a modelling exercise in its own right (the mediating relations are many-to-many, so the aggregation rule is a choice, not a formality) and is out of scope for this submission; we position it as future work in Section 9.3 rather than as a pending fix.

## 5. Failure-Impact Analysis via Heterogeneous GNN and Interpretable Forecasting

Quality attribution (Section 4) tells an architect why a component is structurally critical. This section asks the complementary question: *how much of the system actually fails* when a given component fails, and how well each predictor — interpretable and learned — anticipates it. We define the three simulation oracles that supply ground truth (Section 5.1), the two predictors we evaluate against them, including the Heterogeneous Graph Transformer architecture (Section 5.2), the independence between predictor inputs and the label path that makes the evaluation sound (Section 5.3), and two analyses that take node type seriously: a direct test of the hypothesized shared-library blast-radius mismatch, which we report as a negative result (Section 5.4), and a stratified-correlation consistency check (Section 5.5).

### 5.1. Ground Truth: Three Simulation Oracles

In the absence of runtime telemetry, ground truth is produced by discrete-event failure simulation over the *raw* structural graph  $G_{\text{structural}}$  — directly on PUBLISHES\_TO, SUBSCRIBES\_TO, ROUTES, RUNS\_ON, CONNECTS\_TO, and USES edges, without the derived DEPENDS\_ON projection. For each component  $v$ , a failure is injected at  $v$ , the resulting disruption is propagated through the topology over a fixed horizon, and the residual service degradation is measured.

**The framework contains three such oracles, and they are not interchangeable.** We name them here rather than later, because which one backs a given number materially bounds what that number can support:

- $I^*(v)$  — produced by **FaultInjector**. The mean subscriber feed-loss fraction under a breadth-first cascade. This is the label the learned predictors are trained and evaluated against, and it backs the predictor tables of Section 8.1.
- $I_{\text{comp}}(v)$  — produced by **FailureSimulator**. A four-component weighted composite,

$$I_{\text{comp}}(v) = 0.35 \text{ reachability\_loss} + 0.25 \text{ fragmentation} + 0.25 \text{ throughput\_loss} + 0.15 \text{ flow\_disruption}, \quad (22)$$

where *reachability\_loss* is the fraction of weighted publisher→topic→subscriber paths broken, *fragmentation* is the post-removal graph-partition severity, *throughput\_loss* is the fraction of topic-weight throughput disrupted, and *flow\_disruption* is the fraction of complete pub→topic→sub flow triples broken. The score is graded in  $[0, 1]$ , and its component weights are stated judgements checked for AHP consistency, on the same footing as those of Section 4.3.  $I_{\text{comp}}$  backs the validation gates, the library and stratified analyses of Section 5.4–Section 5.5, and the remediation acceptance test of Section 6.4.

- $I_{\text{dyn}}(v)$  — produced by `MessageFlowSimulator`. The drop in delivered message rate that the *surviving* consumers experience when  $v$  fails,

$$I_{\text{dyn}}(v) = \text{delivery\_rate}_{\text{before}} - \text{delivery\_rate}_{\text{after}}, \quad (23)$$

obtained not by traversing edges but by discrete-event simulation of the actual traffic: each publisher emits at its topic’s declared rate, every topic fans out into a bounded per-subscriber queue, and the fault is injected mid-run. Both windows exclude the faulted node’s own receipts, and a silenced publisher’s unmet demand stays in the denominator, so a component is credited only with the damage it does to *others*.  $I_{\text{dyn}}$  trains nothing and gates nothing: it is reported in Section 7.5 as a construct-validity check on the other two, and is used for no other purpose in this paper.

$I^*$  and  $I_{\text{comp}}$  agree only weakly — mean Spearman  $\rho = 0.394$  across the seven scenarios (Section 7.5). We therefore treat evidence gathered against one as *not* transferring to a claim measured against the other, and apply that constraint to our own analyses rather than leaving it implicit; Section 7.5 quantifies the agreement and the label-coverage bounds, and Section 8.2 flags where the distinction bites. Where a statement below holds for either label-producing oracle, we write simply “the simulated labels”.

**Cascade propagation.** The two cascade oracles share the propagation semantics. A subscriber becomes eligible to fail and propagate only once its average feed loss reaches a `propagation_threshold` (default 0.2); below the threshold, partial feed loss is treated as recoverable degradation rather than a cascade trigger. Broker failure yields continuous per-topic feed loss  $L(t) = |\text{failed\_routers}(t)|/|\text{all\_routers}(t)|$ , correctly modeling multi-broker redundancy. Because intra-wave propagation order is tie-broken stochastically, each scenario is run over multiple seeds; impact is reported as the across-seed mean with its standard deviation, the latter itself a fragility signal at cascade boundaries and the noise scale the remediation filter of Section 6.4 is calibrated against.

## 5.2. Two Predictors over the Same Model

We evaluate two predictors of simulated cascade impact, deliberately spanning the interpretability–capacity spectrum:

- **Interpretable predictor.** The composite quality score  $Q(v)$  of Section 4, computed deterministically on  $G_{\text{analysis}}$  with no learned parameters. Its ranking of components is taken directly as a criticality prediction.
- **Learned predictor.** A **Heterogeneous Graph Transformer (HGT)** that assigns relation-specific attention and message-passing parameters across the five node types (App, Broker, Topic, Node, Library) and six `DEPENDS_ON`/structural edge types, so that message transformations differ by the semantic

relation they traverse rather than being shared across a flattened graph. The *HGL-QoS* variant additionally injects the continuous QoS attributes  $(r, d, p)$  from Section 3.2) directly into the edge-attention aggregation, scaling message magnitude by interface contract strength; the base **HGL** variant masks these QoS fields to isolate the contribution of typing alone from the contribution of QoS encoding (RQ3, Section 8.3). Both variants consume features from the structural analysis result  $G_{\text{analysis}}$  (not the simulator) and are trained inductively against the  $I^*(v)$  labels of Section 5.1.

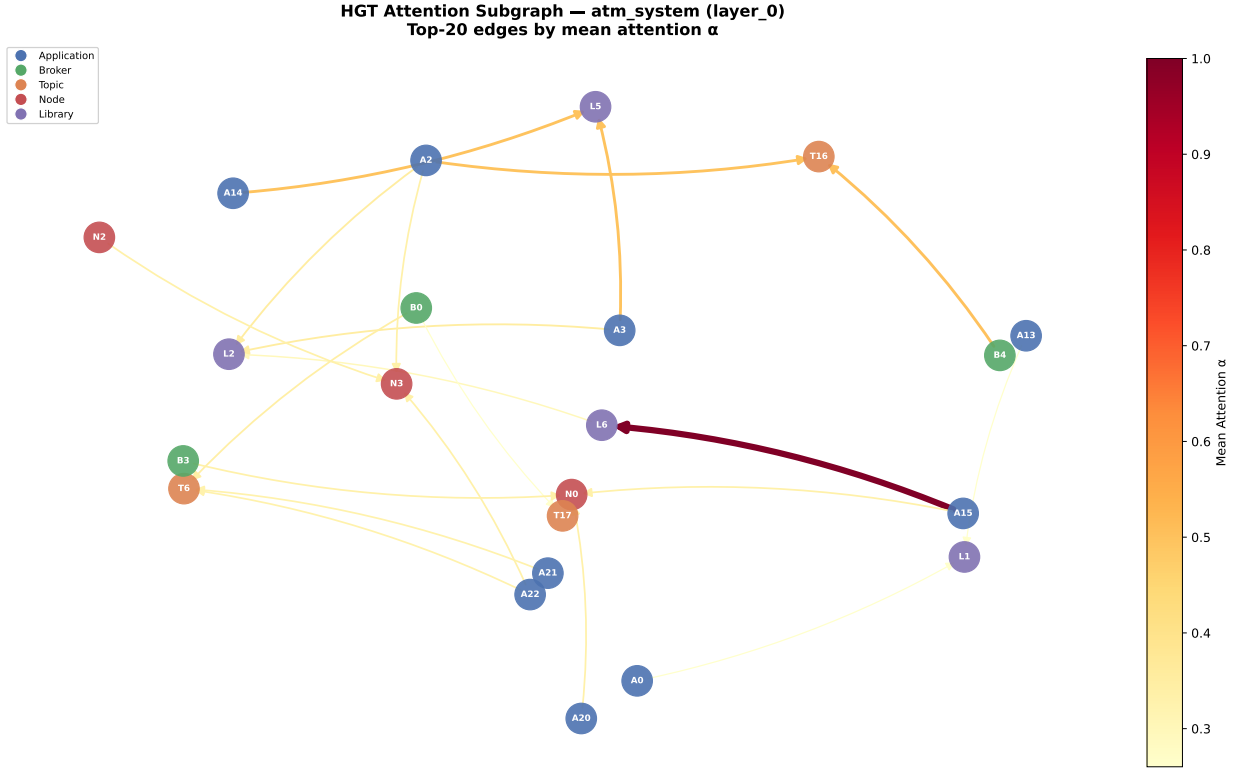


Figure 3: learned relation-specific attention over a case-study subgraph, with per-edge  $\alpha$  from the HGT layer’s own softmax.

For the purpose of RQ1 we report the two predictors separately rather than blended, so that the question — *where* does typed learning improve on the interpretable score, and does that answer depend on which metric is asked about? — is settled on like-for-like rankings.

### 5.3. The Independence Guarantee

The evaluation is only meaningful if the predictor cannot see its own labels. Two structural properties enforce this. First, the predictors operate on  $G_{\text{analysis}}$  (the derived **DEPENDS\_ON** projection and its structural metrics), whereas both simulators operate on  $G_{\text{structural}}$  (the raw edges); the label-producing computation and the feature computation are therefore distinct passes over distinct graph views. Second, no simulation



output — reachability, fragmentation, throughput, or flow disruption — is ever fed back as an input feature to  $Q(v)$  or to the learned predictor. Consequently, a measured correlation between a predictor and the simulated labels, under either oracle, reflects genuine predictive content rather than leakage, which is the property that licenses the framework’s pre-deployment claim. For the learned predictor specifically, the second property is also checked at inference time rather than only held as a design invariant: the GNN service raises before running a forward pass if a feature tensor carries a target label attribute, if any node type’s feature width collides with the label dimensionality, or if a label key appears among the input keys — a defensive check against a leakage bug introduced later in this codebase’s lifetime, not a proof that no such bug exists today. The same discipline governs the remediation stage (Section 6): its candidate-generation phase never reads simulated impact.

#### 5.4. The Shared-Library Blast Mechanism: A Negative Result

Shared libraries have a structurally distinctive failure mode (Section 3.3, Rule 5): a *simultaneous* blast rather than a sequential cascade, in which every consuming application fails in one event rather than along a propagation path. This is invisible to topology-only centrality, which sees an ordinary node of ordinary degree, and it motivated a specific hypothesis — that a library’s composite score  $Q(v)$  would understate its true cascade impact, producing a moderate- $Q$ /near-total- $I$  mismatch. This section is measured throughout against  $I_{\text{comp}}(v)$  (Section 5.1).

We tested this directly across all seven synthetic scenarios (165 Library-type nodes in total) and **did not find the hypothesized mismatch**. The highest composite score reached by any library in the suite is  $Q = 0.422$  (a library with 4 consuming applications), well short of the  $Q \approx 0.5$  region the hypothesis anticipated, and its simulated impact is modest ( $I_{\text{comp}} = 0.086$ ). More importantly, across every library in the corpus,  $I_{\text{comp}}(v)$  never exceeds  $Q(v)$ : the composite score is, if anything, mildly conservative (over-cautious) relative to simulated impact for this node type, not blind to a hidden risk. The clearest low- $Q$  case with substantial fan-out — a library with 12 consuming applications — still has  $I_{\text{comp}} = 0.119$  against  $Q = 0.255$ . Nor does any single-node failure in the suite approach a near-total impact: the largest composite impact from failing any one component, of any type, across all seven scenarios, is  $I_{\text{comp}} = 0.320$  (an infrastructure node), roughly a third of the magnitude the blast-radius hypothesis anticipated.

We report this as a negative result rather than omit it. Two readings are consistent with the data. First, the mechanism itself — simultaneous, type-specific failure via Rule 5 — remains a real structural distinction worth preserving in the model (Section 3.3, Section 4.6), independent of whether it produces a large low- $Q$ /high- $I$  gap in *this* suite; a typed model that can represent the mechanism is not obligated to find a dramatic instance of it in every corpus. Second, the seven synthetic scenarios evaluated here may simply under-represent topologies with a genuinely high-fan-out, low-redundancy shared library — a gap between what a model can express and what a given benchmark suite happens to exercise. We do not claim

779 to have distinguished between these readings, and we retain the FanOutReduction operator’s blast-radius  
780 trigger (Section 6.3) as a structurally motivated safeguard rather than as a mechanism validated by this  
781 particular empirical result.

#### 782 5.5. Stratified Correlation: A Consistency Check

783 A single pooled correlation between predicted criticality and simulated impact, computed over all node  
784 types at once, can in principle be misleading if node types occupy sufficiently different regions of the  
785  $(Q, I)$  plane: pooling heterogeneous populations with divergent conditional relationships can produce a  
786 Simpson’s-paradox-style near-zero aggregate that conceals strong within-type correlations. We checked for  
787 this directly, against  $I_{\text{comp}}(v)$ . Pooling  $(Q, I_{\text{comp}})$  pairs across all seven scenarios (1,545 nodes), the pooled  
788 Spearman correlation is  $\rho = 0.374$  ( $p \approx 2.2 \times 10^{-52}$ ). Computed separately by node type, the correlations  
789 are: Broker  $\rho = 0.429$  ( $n = 36$ ), InfraNode  $\rho = 0.409$  ( $n = 119$ ), Library  $\rho = 0.351$  ( $n = 165$ ), Application  
790  $\rho = 0.346$  ( $n = 850$ ), Topic  $\rho = 0.322$  ( $n = 375$ ) — all significant at  $p < 0.01$ .

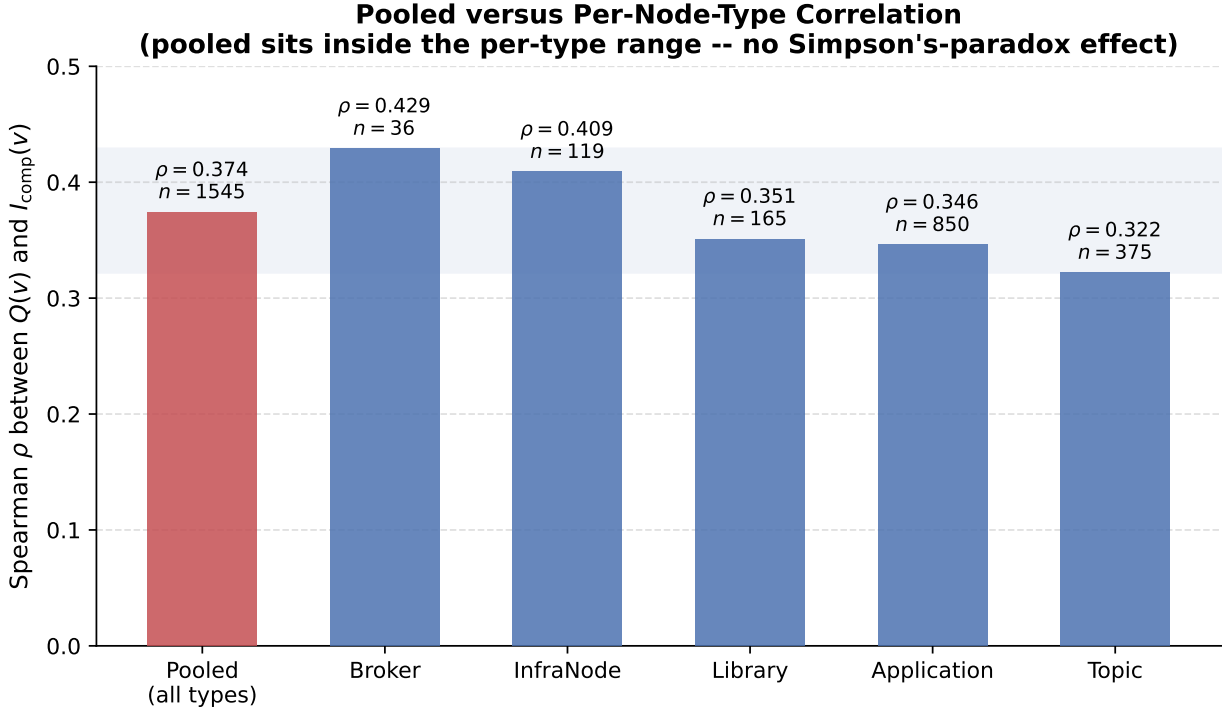


Figure 4: pooled versus per-node-type Spearman  $\rho$  between  $Q(v)$  and  $I_{\text{comp}}(v)$ , with per-type sample sizes.

791 **We do not find a Simpson’s-paradox effect in this suite:** the pooled figure (0.374) sits inside the  
792 per-type range (0.322–0.429) rather than diverging sharply from it. This is nonetheless a useful result, not a  
793 null one. It confirms that the predictive relationship between  $Q(v)$  and simulated impact is of consistent,  
794 moderate strength across every component type — the framework is not quietly failing on some types while

succeeding on others in a way a pooled figure would hide — and it validates stratified reporting as good practice even where it happens not to overturn the pooled conclusion. We report correlation *by node type* throughout (Section 8) on that basis, rather than because pooling was shown to be actively misleading here.

**Three scoping conditions on this check.** First, it is computed against  $I_{\text{comp}}(v)$ , whereas the predictor tables in Section 8.1 are computed against  $I^*(v)$ ; the two oracles agree at mean  $\rho = 0.394$  (Section 7.5), so this consistency check does not transfer to those tables. Second, the check was worth running on its own terms: the effect it looked for *does* occur elsewhere in this study. In the predictor evaluation, pooling Application and Library nodes into a single correlation moved HGL on `av_system` from  $\rho = 0.836$  within Applications to 0.46 pooled — a case where a pooled figure was actively misleading, and one that went unnoticed until the evaluation contract of Section 7.3 was imposed. The methodological point stands independently of the negative finding here.

Third, and cutting across both: the figures in this subsection aggregate components drawn from seven different systems, while D4 (Section 4.1) makes  $Q(v)$  comparable only *within* a system’s own score distribution. The aggregate is therefore a diagnostic over the union of seven within-system rankings, not a criticality measurement over a single population, and it is reported here only to answer the narrow question it was built for — whether the  $Q$ – $I$  relation holds at similar strength in every component type. It should not be read as a cross-system criticality result, and no claim elsewhere in the paper rests on the pooled value.

---

## 6. Prescriptive Remediation and CI/CD Quality Gating

Attribution (Section 4) and impact analysis (Section 5) are diagnostic: they tell an architect *which* components to harden and *why*. This section closes the loop with a prescriptive stage that proposes concrete architectural edits and verifies that they actually reduce simulated failure impact, before any deployment. The stage is designed to preserve the same independence discipline as the rest of the framework: candidate edits are generated from structure alone, and only a separate simulation pass decides whether to accept them. The section then describes how the diagnostics are operationalised as a continuous, delta-aware CI/CD quality gate (Section 6.6).

### 6.1. A Two-Phase Generate–Verify Procedure

Remediation runs in two strictly separated phases.

**Generate.** Given the structural model  $G_{\text{analysis}}$  and its attribution, a set of operators (Section 6.2) propose candidate topology edits — each a small, concrete modification such as adding a replica or an alternative route. Generation reads only structure: component types, the derived `DEPENDS_ON` graph, and structural blast-radius signals. It never reads simulated impact.

**Verify.** Each candidate edit  $e$  is applied to produce a counterfactual graph  $G' = e(G)$ , on which the **FailureSimulator** of Section 5.1 is re-run from scratch. The edit is accepted only if it reduces  $I_{\text{comp}}$  by a robust margin (Section 6.4). This stage is therefore measured against  $I_{\text{comp}}$  throughout, not against the  $I^*$  labels behind the predictor tables of Section 8.1 — a scoping condition that follows from the weak agreement between the two oracles (Section 5.1, Section 7.5). Verification is an oracle check against ground truth, not against the score that proposed the edit.

This separation matters: a stage that both proposed and scored edits using the same signal would be optimizing against itself. By generating from structure and verifying by simulation, the stage cannot manufacture an apparent improvement that the simulator does not confirm.

## 6.2. Remediation Operators

Four operators formalize the framework’s existing heuristic recommendations (SPOF redundancy, alternative routing for bridges, fan-out reduction for over-subscribed topics, decoupling of multi-topic pairs) into verifiable edits. Each is keyed to a structural trigger and targets a specific failure mode:

Table 12: The four remediation operators, their structural triggers, and the failure mode each targets.

| Operator                    | Structural trigger                                                              | Edit applied                                                            | Failure mode targeted                  |
|-----------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------|
| <b>RedundancyInsertion</b>  | Isolated articulation point / high A SPOF                                       | add a redundant instance or redistribute responsibilities               | graph-partitioning SPOF                |
| <b>PathDiversification</b>  | bridge edge / single routing path for a topic                                   | add an alternative route (e.g. a second routing broker or network link) | fragmentation on a non-redundant edge  |
| <b>FanOutReduction</b>      | high structural blast radius (topic subscriber fan-out; library consumer count) | interpose an intermediary or split the over-shared channel              | simultaneous blast / fan-out explosion |
| <b>SharedTopicReduction</b> | high multi-path coupling (large <code>path_count</code> / MPCl between a pair)  | decouple redundant shared topics between the pair                       | multi-channel coupling fragility       |

The operators span the RMAV dimensions deliberately: **RedundancyInsertion** and **PathDiversification** address Availability, **FanOutReduction** addresses Reliability (blast radius), and **SharedTopicReduction** addresses Maintainability coupling.

## 6.3. Triggering on Blast Radius, not on $Q(v)$

**FanOutReduction** is the operator that connects remediation to the hypothesis tested in Section 5.4, and its trigger is deliberately *not* the composite  $Q(v)$ . A shared library or an over-subscribed topic could in

principle carry only a moderate  $Q$  while nonetheless dominating simultaneous-blast impact; triggering on  $Q$  would then skip exactly the components most worth remediating. Instead, FanOutReduction fires on direct structural blast-radius signals — subscriber fan-out for topics, consumer count for libraries — so that a low- $Q$ , high-blast component is still selected for a candidate edit. This is the remediation-side expression of the paper’s central claim that single-score criticality is insufficient: the *attribution* exposes the gap, and the *operator* is designed not to fall into it.

This is a statement about the trigger’s design, not about its yield. Section 5.4 finds no low- $Q$ /high- $I$  library population in this suite for the trigger to catch, and Section 6.7 shows that its yield is concentrated in the two topologies that actually contain a fan-out bottleneck. We retain the structural trigger because triggering on  $Q$  would be unsound if such a component existed, not because we have shown that one does.

#### 6.4. Acceptance Criterion

An edit should do more than nudge the mean impact down; it should improve impact by a margin that exceeds the simulator’s own seed noise. For a candidate edit producing  $G'$ , let  $\Delta I = I_{\text{comp}}(v; G) - I_{\text{comp}}(v; G')$  be the reduction in simulated impact over the components present in both graphs, and let  $\sigma_{\text{seed}}$  be the across-seed standard deviation of that reduction (Section 5.1). The acceptance rule is

$$\Delta I > \kappa \sigma_{\text{seed}} \quad \text{for every sampled } \texttt{propagation\_threshold}, \quad (24)$$

evaluated **per candidate edit, on its own counterfactual graph**, before the edit is committed to the policy. Two design choices are load-bearing. First, normalising by  $\sigma_{\text{seed}}$  ties the bar to the fragility of the cascade at that point, so an edit is accepted only when its benefit is distinguishable from propagation-order noise. Second, requiring the inequality to hold across the full **propagation\_threshold** sweep makes acceptance robust to the threshold’s value — which Section 8.3 shows is not a benign parameter, since  $\rho$  against ground truth spans 0.230 across its range.

**PrescribeService** implements this as a three-phase procedure: compile the candidate policy (Section 6.2), verify each candidate independently by constructing a graph containing that edit alone and re-simulating it across thresholds and seeds, then apply only the accepted subset and measure the System Risk Index before and after on the mutated graph as a whole (Section 6.7’s Table 13). Each candidate carries its measured  $\Delta I$ ,  $\sigma_{\text{seed}}$  and — when rejected — the threshold at which it failed, so a run reports what it declined and why rather than only what it applied.

**What this replaces.** An earlier version of this framework compiled a policy and applied all of it unconditionally, judging the result by a single end-state check. Under that design an edit that made the system worse could be carried by edits that made it better, which is the mechanism behind the mixed aggregate previously reported in Section 6.7. Per-edit verification removes that

failure mode by construction: a regressing edit is rejected individually and never reaches the mutated graph.

An empty accepted set is a valid outcome, not a failure, and is reported as such rather than as a no-op mutation with an unchanged risk index. On small topologies it is common for no candidate to clear the bar — which is the filter working, and is more informative than a policy applied on the strength of an unverified aggregate.

### 6.5. Independence Invariants

The stage obeys three invariants that mirror the predictor/simulator separation of Section 5.3:

1. **Generate never reads  $I_{\text{comp}}(v)$ .** Candidate edits come from structure and attribution only.
2. **Verify re-invokes the canonical simulator** on  $G'$  from scratch, rather than estimating the counterfactual impact from the predictor. The re-simulation is performed *per candidate edit*, on a graph containing that edit alone, across the propagation-threshold sweep and the seed set; only the accepted subset is then applied and re-checked at system level (Section 6.7).
3. **No Verify result feeds back into Generate within a run.** There is no closed-loop search that would let simulated impact influence which edits are proposed, which would reintroduce the circularity the framework is built to avoid.

Together these keep the diagnostic and evaluation signals separate: the thing that proposes a fix and the thing that measures it are never the same signal, so an edit is admitted only on evidence the proposing signal did not produce.

### 6.6. CI/CD Quality Gate Implementation

To operationalise these diagnostics, SaG integrates into developer workflows as a blocking Quality Gate in the CI/CD pipeline. When a pull request introduces configuration or architecture modifications (Architecture-as-Code changes), the pipeline executes the analyzer via a dedicated CLI script, `detect_antipatterns.py`, which runs the full anti-pattern catalog against the candidate topology and issues an exit code.

**Exit-code protocol.** The gate issues exit codes that govern pipeline execution: - **Exit Code 0:** No architectural anomalies found; deployment is permitted. - **Exit Code 1:** Medium-severity architectural smells found (e.g., chatty pairs or QoS mismatch warnings); deployment is permitted with warnings. - **Exit Code 2:** CRITICAL or HIGH severity anomalies found (e.g., single points of failure, cyclic dependencies, or broker overload); the build is broken and **deployment is blocked**.

This is an *absolute* gate: every run evaluates the candidate topology’s full finding set, not a diff against a prior baseline. It has a known consequence, which we do not paper over: a real architecture that carries an intentional, risk-accepted single point of failure — a sole-source surveillance feed, a deliberately unreplicated

legacy broker — fails the build on every commit, indistinguishable from a genuine regression. A *delta-aware* gate that evaluates the candidate against a merge-base topology and blocks only on newly introduced findings, together with a waiver register recording accepted risk (entity, rule, expiry) so it stays visible rather than silently re-triggering, would close this gap; we describe the design in Section 9.3 as future work rather than claim it here, since the mechanism is not implemented in the released tool.

The underlying analysis-and-detection machinery is in-memory and does not require a live database connection — `saag`’s thread-safe `MemoryRepository` port satisfies the same repository interface `detect_antipatterns.py` consumes, and is what the timing harness behind Section 8.4’s measurements uses. Wiring that path into `detect_antipatterns.py` itself, so the packaged CLI does not require a Neo4j connection during a CI build, is a small remaining integration step we have not made; today the script connects to a running database.

### 6.7. What Remediation Yields Under Per-Edit Verification

Running the full Generate→Verify procedure of Section 6.4 across the scenario suite, with  $\kappa = 1.0$ , three propagation thresholds  $\{0.1, 0.2, 0.5\}$  and the first three of the five canonical seeds  $\{42, 123, 456\}$ , gives the following. The seed set is reduced here, and only here, because acceptance requires a full re-simulation *per candidate edit per threshold*, so the sweep over 332 candidates is the most expensive experiment in the study. The reduction is a compute concession, not a methodological one, and we flag the consequence rather than argue it away:  $\sigma_{\text{seed}}$  is the quantity the acceptance rule divides by, and a three-seed estimate of it is noisier than a five-seed one, which makes the filter correspondingly less reliable at the margin. “Cand.” is the number of edits the generator proposed; “Acc.” the number that cleared  $\Delta I > \kappa \sigma_{\text{seed}}$  at *every* threshold;  $\Delta\text{SRI}$  is the system risk index change from applying the accepted subset (positive = risk reduced).

Table 13: Remediation yield under per-edit verification,  $\kappa = 1.0$ , thresholds  $\{0.1, 0.2, 0.5\}$ , seeds  $\{42, 123, 456\}$ .

| Scenario                      | Baseline SRI  | Mutated SRI   | $\Delta\text{SRI}$ | Cand.      | Acc.      | Rej.      |
|-------------------------------|---------------|---------------|--------------------|------------|-----------|-----------|
| Autonomous Vehicle            | 0.3645        | 0.3615        | +0.0030            | 35         | 3         | 32        |
| IoT Smart City                | 0.4260        | 0.4102        | <b>+0.0158</b>     | 58         | 38        | 20        |
| Financial Trading             | 0.3842        | 0.3785        | +0.0057            | 31         | 5         | 26        |
| Healthcare                    | 0.3809        | 0.3784        | +0.0025            | 19         | 14        | 5         |
| Hub-and-Spoke                 | 0.3576        | 0.3502        | +0.0074            | 30         | 14        | 16        |
| Microservices Mesh            | 0.3612        | 0.3577        | +0.0035            | 40         | 19        | 21        |
| <b>Hyper-Scale Enterprise</b> | <b>0.3614</b> | <b>0.3475</b> | <b>+0.0139</b>     | <b>119</b> | <b>69</b> | <b>50</b> |

By parallelizing counterfactual verification across multi-core CPU worker pools (`ProcessPoolExecutor`), we evaluate the per-edit acceptance filter across all seven benchmark scenarios, including Hyper-Scale

Enterprise (350+ components, 119 candidate edits). Across the full 7-scenario suite, 162 of 332 candidate edits (48.8%) clear the multi-threshold acceptance filter ( $\Delta I > \kappa \sigma_{\text{seed}}$ ). On Hyper-Scale Enterprise, 69 of 119 candidates clear the filter, reducing System Risk Index from 0.3614 to 0.3475 ( $\Delta \text{SRI} = +0.0139$ ).

**Per-edit verification prevents regressing edits.** Every admitted edit is guaranteed to reduce cascade impact individually across all propagation thresholds. This removes by construction the failure mode of the previous unverified design, in which a regressing edit could be carried by an improving one — an aggregate that was arithmetically correct and substantively misleading. Under the per-edit filter no scenario in the suite regresses.

**Individually-verified edits are not shown to compose.** Acceptance is decided on singletons: each candidate is simulated alone, on a graph containing only that edit. Nothing in the procedure establishes that a set of individually-accepted edits remains beneficial when applied together, and the  $\Delta \text{SRI}$  column reports the outcome of applying the accepted subset rather than a verified prediction of it. Verifying *subsets* rather than singletons would close this at combinatorial cost; we note it as a limitation of the current design rather than claiming compositional safety we have not tested.

**The acceptance rate varies widely with topology, and that is the substantive finding.** The filter admits 3 of 35 candidates on Autonomous Vehicle and 5 of 31 on Financial Trading, but 38 of 58 on IoT Smart City and 14 of 19 on Healthcare. The two largest absolute risk reductions come from IoT Smart City ( $\Delta \text{SRI} = +0.0158$ ) and Hyper-Scale Enterprise (+0.0139) — the two scenarios with pronounced hub-topic and fan-out structure. That the operator set has purchase precisely where a fan-out bottleneck exists is consistent with how the operators are defined (Section 6.2), and suggests the honest scope for this stage is narrower than “topology-level hardening”: it is closer to “fan-out decomposition where a fan-out bottleneck actually exists”. Across the suite the improvements are real but small in absolute terms — between +0.0025 and +0.0158 SRI — which is the result we report rather than a demonstration that the prescriptive stage is yet practically valuable (Section 9.3).

## 7. Experimental Setup

This section describes the data, predictors, metrics, and protocols used to answer RQ1–RQ4 (Section 8). The design follows one overriding principle, carried from the framework’s independence guarantee (Section 5.3): every predictor is evaluated against the same simulator-derived ground truth produced by an independent process, so the claims we make are *comparative* — which modeling choices perform better under identical conditions — rather than assertions of absolute accuracy in operational deployments.

### 7.1. Datasets

**Synthetic suite.** We evaluate on seven synthetic pub-sub scenarios spanning distinct deployment domains — autonomous vehicles, high-frequency trading, clinical healthcare integration, centralized hub-and-



spoke enterprise systems, distributed IoT smart-city telemetry, cloud-native microservices, and large-scale enterprise pub-sub. The scenarios are produced by a statistical topology generator and range from 50 to 300 applications per scenario, exercising fan-out-dominated, dense-pub-sub, and anti-pattern/SPOF regimes with different dominant failure mechanisms.

**Real-world open-source suite.** To test operational generalizability on authentic software graphs, we evaluate SaG on three real-world open-source software architectures: 1. **Autoware.universe (ROS 2 Autonomous Driving Platform) [47]:** An authentic cyber-physical ROS 2 pub-sub architecture comprising 32 Applications (perception, sensing, localization, planning, control), 24 Topics with explicit DDS QoS profiles (RELIABLE/BEST\_EFFORT, TRANSIENT\_LOCAL/VOLATILE), 3 Brokers (CycloneDDS, FastDDS, Zenoh), 6 Deployment Nodes, 10 Shared C++ Libraries (autoware\_universe\_utils, tier4\_aware\_utils), and realistic SonarQube code metrics. 2. **Production Cloud-Native Microservices Mesh [48]:** An authentic microservice architecture based on the Google Online Boutique e-commerce benchmark, comprising 22 Microservices (order, payment, inventory, auth, analytics, notifications), 20 Topics across Kafka, RabbitMQ, Redis PubSub, and NATS, 6 Kubernetes/Cloud nodes, and 8 shared helper libraries. 3. **Train-Ticket Railway Booking Mesh [49]:** An authentic microservice architecture based on the Fudan University Train-Ticket benchmark [49], comprising 41 Microservices (order, travel, preserve, route, seat, payment, food, security, admin), 30 Topics, 3 Brokers (RabbitMQ, Redis PubSub, Spring Eureka naming server), 8 deployment Nodes, and 8 shared Spring/MyBatis libraries. At 90 components it is the largest of the three real-world graphs.

Pooled across all synthetic and real-world scenarios, the evaluation corpus exercises 1,770 components. Every scenario is versioned in the replication package. The synthetic suite is registered in a manifest carrying a canonical SHA-256 per dataset, and a regression test re-generates each synthetic dataset from its configuration and fails on any divergence, so the corpus is byte-reproducible from its configs. The three real-world graphs are not part of that manifest: they are versioned files produced by a hand-written adapter rather than by the statistical topology generator, so the byte-identity guarantee applies to the synthetic suite only.

All seven scenarios are used for the predictor evaluation (Section 8.1–Section 8.3), the analyses of Section 5.4–Section 5.5, and the remediation evaluation of Section 6.7, the last of which became tractable at the largest scale by parallelising counterfactual verification across candidates.

## 7.2. Predictors and Baselines

The evaluation compares predictors spanning the interpretability–capacity spectrum, all consuming the same structural analysis of each scenario:

Table 14: Predictors and baselines, and the factor each contrast isolates.

| Predictor                        | Description                                                               | Role                               |
|----------------------------------|---------------------------------------------------------------------------|------------------------------------|
| <b>RMAV</b> / $Q$                | deterministic multi-dimensional composite<br>(Section 4)                  | interpretable<br>predictor         |
| <b>HGL</b>                       | heterogeneous graph transformer,<br>QoS-masked                            | learned predictor<br>(typed)       |
| <b>HGL-QoS</b>                   | heterogeneous graph transformer,<br>QoS-encoded                           | learned predictor<br>(typed + QoS) |
| <b>GL</b> / <b>GL-QoS</b>        | homogeneous GAT on the type-collapsed<br>projection                       | learning baseline<br>(untyped)     |
| <b>Topo-BL</b> / <b>Topo-QoS</b> | structural centrality (betweenness,<br>articulation points; QoS-weighted) | non-learning<br>baseline           |

The contrast **Topo-\*** vs learned isolates the value of learning (RQ1); **GL** vs **HGL** isolates the value of *typed* heterogeneity; **HGL** vs **HGL-QoS** isolates the value of explicit QoS encoding (RQ3); and **RMAV/Q** vs the learned predictors isolates when interpretable attribution suffices. The structural baselines’ features are kept decoupled from the GNN inputs so that no comparison leaks information across the predictor boundary.

### 7.3. Evaluation Metrics

We report metrics in three families, plus the stratification and significance machinery:

- **Ranking.** Spearman rank correlation  $\rho$  between predicted criticality and  $I^*(v)$  is the primary metric, complemented by NDCG@10 and Top-5/Top-10 overlap for the practically relevant case in which only a few components can be hardened.
- **Identification.** Precision, recall, and F1 for critical-component detection, plus SPOF-F1 for articulation-point classification against simulated availability impact.
- **Regression.** RMSE and MAE between predicted and simulated scores, for calibration.
- **Stratified reporting.** Following the consistency check of Section 5.5,  $\rho$  is always reported *by node type* in addition to (not instead of) any pooled figure.
- **Statistical rigor.** Bootstrap 95% confidence intervals [50] ( $B = 2000$  resamples) on mean  $\rho$  [51], and paired Wilcoxon signed-rank tests [52] ( $p < 0.05$ ) for predictor comparisons across scenarios.

**Validation gate thresholds.** The implementation carries two independent gate registries — a fixed nine-check one and the topology-class-adjusted five-check one below; Table 15 and Section 8.5 document

only the latter, which is the one the reported evaluation runs against. It is parameterised by topology class, because a threshold that discriminates on a dense topology is either trivial or unattainable on a sparse one. The default targets are  $\rho \geq 0.70$  and  $F1 \geq 0.80$ ; the **sparse** class, which covers every graph in Section 8.5, applies the five checks below. We tabulate them because Section 8.5 reports gate *failures*, and a reader cannot evaluate a failure against an unstated bar:

Table 15: Topology-class validation gate thresholds. A graph’s class is assigned from the density and hub ratio of its structural graph. All five checks must pass for the gate to pass.

| Check                                       | sparse      | medium      | dense       | hub_spoke   |
|---------------------------------------------|-------------|-------------|-------------|-------------|
| Spearman $\rho$ vs simulated impact         | $\geq 0.75$ | $\geq 0.80$ | $\geq 0.82$ | $\geq 0.85$ |
| $F_1@K$ (critical-set overlap)              | $\geq 0.65$ | $\geq 0.70$ | $\geq 0.72$ | $\geq 0.75$ |
| SPOF-F1 (articulation points vs $I > 0.3$ ) | $\geq 0.60$ | $\geq 0.65$ | $\geq 0.65$ | $\geq 0.70$ |
| FTR, false target rate (lower is better)    | $\leq 0.30$ | $\leq 0.25$ | $\leq 0.25$ | $\leq 0.20$ |
| Predictive gain over degree centrality      | $\geq 0.02$ | $\geq 0.03$ | $\geq 0.03$ | $\geq 0.03$ |

FTR is the fraction of components predicted highly vulnerable ( $V(v) > 0.60$ ) whose simulated reachability impact is nonetheless negligible ( $< 0.10$ ) — a precision check on the Vulnerability dimension specifically. Note that the class adjustment is not uniformly a tightening relative to the  $\rho \geq 0.70$  /  $F1 \geq 0.80$  defaults:  $\rho$  rises to 0.75 on the sparse class, whereas the F1-family thresholds are *relaxed*, because on a sparse graph a handful of components carries the whole signal and a single disagreement moves F1 sharply. We tabulate the rule rather than describe it as a tightening throughout.

**One evaluation contract, one sample.** Every variant in every table is scored by the same function on the same node set. This is a correction rather than a description of prior practice: an earlier version of this study scored the two predictor families on different populations and different samples, and correcting it raised every learned variant by 0.35–0.48  $\rho$  in-distribution while leaving the baselines essentially unchanged (Section 9.2). Three properties now hold:

1. **The evaluation key set is a function of the graph and the labels only** — never of any variant’s predictions — so all variants in a cell see an identical sample. The node population is an explicit, recorded parameter (**application** by default, matching the claim that topology predicts *application-layer* criticality).
2. **The reported figure is held-out.** All variants share one train/validation/test split pinned by node identity. A full-population score flatters a trained model by including the nodes it was fitted on while

leaving a training-free baseline unchanged; the transductive figure is retained separately rather than reported as the headline.

3. **A variant that cannot cover the declared population fails loudly** rather than silently shrinking the sample to a per-variant subset.

**Absent is not zero.** A stratum whose predictions or labels are constant has an *undefined* rank correlation and is reported as such, never as 0.0. This matters for coverage: Topic and physical Node components carry no simulated ground truth at all (Section 7.5), and reporting them as 0.0 presented a labelling gap as a measured model failure.

#### 7.4. Protocols

Two evaluation regimes are used, each answering a different generalization question.

**In-distribution (per-scenario).** For each scenario, predictors are computed and compared against that scenario’s simulated ground truth. This is the regime for RQ1 and RQ2 (Section 8): it asks how well the attribution and learned predictors recover the criticality ordering of a *known* system.

**Inductive (Leave-One-Scenario-Out).** To test generalization to *unseen* architectures — the true pre-deployment condition — we use Leave-One-Scenario-Out (LOSO) cross-validation, which closes the transductive-leakage gap for the learned predictor. For each held-out scenario  $k$ , the model is trained on the remaining six scenarios (with the largest by  $|V|$  used for early stopping) and evaluated on  $k$ , whose nodes never participate in any forward pass and whose labels never enter any loss. Results are aggregated as per-fold mean  $\pm$  std across seeds, then cross-fold mean  $\pm$  std, with per-node-type  $\rho$  retained.

**Multi-seed.** Every configuration is run over five seeds  $\{42, 123, 456, 789, 2024\}$ ; reported scores are seed means, and the across-seed standard deviation  $\sigma_{\text{seed}}$  is both reported and reused as the noise scale in the remediation acceptance criterion (Section 6.4). The one exception is the remediation sweep of Section 6.7, which uses the first three of these seeds for the compute reason stated there.

#### 7.5. Ground Truth: Three Oracles, and What They Can Each Support

The three oracles introduced in Section 5.1 are constructed differently, measure different quantities, and are not interchangeable; conflating them is the most likely way to over-read a result in this paper. This section fixes which analysis rests on which, quantifies how far they agree, and states the label-coverage bounds that apply to each.

Table 16: The three simulation oracles, what each measures, and which results rest on which.

| Symbol               | Engine                      | Quantity                                                                                           | Used for                                                                                                              |
|----------------------|-----------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| $I^*(v)$             | <b>FaultInjector</b>        | Mean subscriber feed-loss fraction under a BFS cascade                                             | Learned-predictor labels; Tables 18 and 20 (Section 8.1); the sensitivity sweeps of Section 8.3                       |
| $I_{\text{comp}}(v)$ | <b>FailureSimulator</b>     | 0.35 reachability + 0.25 fragmentation + 0.25 throughput + 0.15 flow                               | Validation gates; the RMAV dimension decomposition; Section 5.4 and Section 5.5; remediation acceptance (Section 6.4) |
| $I_{\text{dyn}}(v)$  | <b>MessageFlowSimulator</b> | Delivery-rate loss suffered by <i>surviving</i> consumers, by discrete-event simulation of traffic | Reported construct-validity check only — no labels, no gates, no tables                                               |

The two cascade oracles run with a step-function blast-semantics propagation scheme (probability 1.0 for library cascade), `propagation_threshold` default 0.2, a 10-epoch horizon, and the five seeds of Section 7.4, {42, 123, 456, 789, 2024}.  $I_{\text{dyn}}$  shares the seed set and runs 60 simulated seconds per component, with the fault injected at the midpoint.

**Measured agreement between the two cascade oracles is weak.** Their scales differ, so only rank agreement is meaningful; across the seven scenarios, mean Spearman  $\rho = 0.394$  and mean top-20% Jaccard = 0.286, ranging from  $\rho = 0.578$  (Enterprise) down to  $\rho = 0.092$  (Hub-and-Spoke, where they are effectively uncorrelated). All seven correlations are positive, which is a weak convergent-validity argument — two differently-constructed simulators do agree directionally, so neither is purely an artifact of its own construction — but at  $\rho \approx 0.39$  it is weak, and we apply the resulting constraint to our own analyses: a result established against one oracle is not evidence for a claim measured against the other. Section 8.2 flags where this bites.

$I_{\text{dyn}}$  agrees with  $I^*$  far more strongly, and — crucially — does not share its worst case.

Mean Spearman  $\rho(I_{\text{dyn}}, I^*) = 0.765$ , minimum 0.548 (Hub-and-Spoke) — against mean 0.394, minimum 0.092 for the two topological oracles above. Hub-and-Spoke is precisely where  $I^*$  and  $I_{\text{comp}}$  collapse to near-independence;  $I_{\text{dyn}}$  still agrees with  $I^*$  there at  $\rho = 0.548$ , its lowest agreement in the cohort but far from uncorrelated. Because  $I_{\text{dyn}}$  reaches this ranking by simulating traffic through queues rather than by traversing `DEPENDS_ON`, the result is evidence of a different kind than Section 7.5’s first finding: it rules out the cascade *algorithm* as the source of  $I^*$ ’s ranking, which the  $I_{\text{comp}}$  comparison alone cannot do (Section 9.2). Top- $K$  membership is the weaker half of this result — mean top-20% Jaccard is 0.316, comparable to the 0.286 of the two topological oracles — so this is corroboration of *ranking*, not of critical-set identification; no  $F_1@K$  claim in Section 8.1 is supported by  $I_{\text{dyn}}$ .

**Label coverage and the noise ceiling.** Three further properties bound what any reported figure can mean. First, the cascade model has no rule expressing the failure of a Topic or of a physical Node, so those types carry no ground truth at all — 30–47% of components per scenario are unlabelled, they are excluded from scoring rather than scored as zero, and predictions for them are never validated. Broker labels are degenerate in three of seven scenarios for a related reason. Second, the three oracles do not cover the same components, so every agreement figure above is computed over the intersection rather than over the scenario.  $I_{\text{dyn}}$  observes only what carries pub-sub traffic: it scores Applications and those Libraries that publish or subscribe in their own right, and records Brokers, physical Nodes, Topics, and purely-consumed Libraries as unmeasured rather than as harmless. On `enterprise_system` that is 349 components against  $I^*$ ’s 360 — it gives up the ten Brokers and one non-publishing Library, and gains nothing  $I^*$  lacks. Third, the labels have a reproducibility ceiling: across seeds, the ground truth agrees with *itself* at test–retest  $\rho$  of 0.807–1.000, and its own top-20% critical set agrees at Jaccard 0.44–1.00 (deterministic: `FaultInjector`’s cascade previously iterated an unordered subscriber set while consuming seeded random draws, so re-running the *same* seed in a different process could still change the label — this has been fixed and is disclosed as an instrument defect in Section 9.2, and the figures here are the post-fix, process-independent ones). No method can exceed the former, and every top- $K$  metric inherits the latter — a reported  $F_1@K$  on `microservices_system`, where the labels’ own set stability is 0.44, should not be read to a precision the labels do not have.

## 7.6. Model Configuration and Implementation

The learned predictors are implemented in PyTorch Geometric [53]. Table 17 fixes every hyperparameter; all variants share it, so the contrasts of Section 7.2 isolate architecture and features rather than tuning budget. The values were fixed before the reported runs and not tuned per scenario — there is no per-scenario hyperparameter search anywhere in this study, which is a deliberate constraint (a search per scenario would leak held-out information under the in-distribution protocol) and also a limitation, since a tuned baseline might close some of the margins in Section 8.1.

Table 17: Learned-predictor configuration. Identical across HGL, *HGL-QoS*, GL and GL-QoS except where the architecture differs by construction.

| Component               | Setting                                                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Convolution             | HGTConv (heterogeneous); GATConv for the homogeneous GL variants                                                                       |
| Layers                  | 3                                                                                                                                      |
| Hidden channels         | 64                                                                                                                                     |
| Attention heads         | 4                                                                                                                                      |
| Dropout                 | 0.2                                                                                                                                    |
| Input projection        | per-node-type linear $\rightarrow$ LayerNorm $\rightarrow$ ReLU                                                                        |
| Output heads            | four RMAV residual MLPs + one composite head, sigmoid-activated                                                                        |
| Optimizer               | AdamW, learning rate $3 \times 10^{-4}$ , weight decay $1 \times 10^{-4}$                                                              |
| LR schedule             | CosineAnnealingWarmRestarts,<br>$T_0 = \max(50, \text{epochs}/4)$ , $T_{\text{mult}} = 2$ , $\eta_{\text{min}} = 0.01 \cdot \text{lr}$ |
| Gradient clipping       | max-norm 1.0                                                                                                                           |
| Epochs / early stopping | 300, patience 30 on validation loss                                                                                                    |
| Node splits             | 60% train / 20% validation / 20% test, pinned by node identity (Section 7.3)                                                           |
| Loss                    | composite MSE + 0.5 multitask + 0.3 ListMLE ranking + 0.1 pairwise margin + 0.1 RMAV consistency                                       |

**Hardware and runtime.** Training and evaluation were run on a single workstation; the LOSO sweep is the dominant cost, at roughly 31 minutes for HGL and 36 for *HGL-QoS* across all folds and seeds, against 5–6 minutes for the homogeneous variants and well under a minute for the training-free baselines. The CI/CD gate measurements of Section 8.4 were taken on the same machine rather than on hosted runner hardware, so they should be read as an order-of-magnitude feasibility result rather than as a calibrated figure for any particular CI provider.

## 8. Results

We answer RQ1 (when interpretable attribution suffices versus when learning is required, Section 8.1) and RQ2 (what multi-dimensional attribution exposes that centrality misses, Section 8.2), then report the ablations and sensitivity analyses that test the robustness of these answers and settle RQ3 (Section 8.3), evaluate the CI/CD quality gate for RQ4 (Section 8.4), and close with the real-world external-validity evidence for RQ5 (Section 8.5). All figures are seed means over  $\{42, 123, 456, 789, 2024\}$ . Bootstrap 95% confidence intervals ( $B = 2000$ ) accompany Table 18, and predictor comparisons are tested with paired Wilcoxon signed-rank tests across scenarios (Table 19). Where an interval or a test is not reported, it is because the underlying per-fold artifact was not retained, and we say so at that point rather than omit it silently.

### 8.1. RQ1 — Interpretable Attribution versus Learning

Every figure in this section is produced by one evaluation contract (Section 7.3): all six variants are scored on an identical held-out node set, drawn from a single train/validation/test split pinned by node identity and shared across variants. The previously reported version of this table did not have that property, and the correction changes its conclusion; Section 7.3 documents what changed and why.

**In-distribution, typed learning leads on the point estimate.** Table 18 reports Spearman  $\rho$  against simulated impact  $I^*(v)$  on the held-out split, averaged over five seeds, with bootstrap 95% confidence intervals ( $B = 2000$ ) and the held-out sample size  $n$  on which each row’s correlations are computed:

Table 18: In-distribution held-out Spearman  $\rho$  against  $I^*(v)$ , seed means over  $\{42, 123, 456, 789, 2024\}$  with bootstrap 95% CIs.  $n$  is the number of held-out Application-type components scored in that scenario (Section 7.3).

| Scenario   | $n$ | Topo-BL       | Topo-QoS     | GL           | GL-QoS       | HGL          | <i>HGL-QoS</i> |
|------------|-----|---------------|--------------|--------------|--------------|--------------|----------------|
| AV         | 16  | 0.308 [0.13,  | 0.750 [0.55, | 0.760 [0.66, | 0.655 [0.40, | 0.713 [0.51, | 0.692 [0.47,   |
| System     |     | 0.46]         | 0.91]        | 0.84]        | 0.88]        | 0.88]        | 0.87]          |
| Enterprise | 60  | 0.393 [0.29,  | 0.797 [0.75, | 0.853 [0.81, | 0.513 [0.21, | <b>0.885</b> | 0.883 [0.84,   |
|            |     | 0.50]         | 0.85]        | 0.89]        | 0.79]        | [0.86, 0.91] | 0.92]          |
| Financial  | 12  | 0.246         | 0.709 [0.58, | 0.851 [0.82, | 0.874 [0.85, | 0.882 [0.85, | <b>0.903</b>   |
| Trading    |     | [-0.05, 0.56] | 0.84]        | 0.89]        | 0.89]        | 0.91]        | [0.88, 0.92]   |
| Healthcare | 10  | -0.182        | 0.772 [0.59, | 0.815 [0.76, | 0.804 [0.74, | 0.842 [0.80, | <b>0.845</b>   |
|            |     | [-0.40, 0.06] | 0.88]        | 0.87]        | 0.85]        | 0.87]        | [0.80, 0.88]   |



| Scenario       | $n$ | Topo-BL                   | Topo-QoS                  | GL                        | GL-QoS                    | HGL                       | <i>HGL-QoS</i>            |
|----------------|-----|---------------------------|---------------------------|---------------------------|---------------------------|---------------------------|---------------------------|
| Hub-and-Spoke  | 14  | 0.299 [0.12, 0.48]        | 0.511 [0.21, 0.76]        | 0.494 [0.21, 0.72]        | 0.475 [0.34, 0.63]        | 0.537 [0.43, 0.65]        | <b>0.557</b> [0.47, 0.65] |
| IoT Smart City | 40  | -0.063 [-0.17, 0.05]      | 0.068 [-0.07, 0.20]       | 0.674 [0.54, 0.81]        | 0.474 [0.27, 0.67]        | <b>0.891</b> [0.87, 0.91] | 0.883 [0.86, 0.91]        |
| Microservices  | 18  | 0.302 [0.07, 0.54]        | 0.556 [0.38, 0.71]        | 0.524 [0.41, 0.63]        | 0.436 [0.31, 0.56]        | 0.362 [0.05, 0.55]        | 0.354 [0.18, 0.53]        |
| <b>Mean</b>    | —   | <b>0.186</b> [0.02, 0.32] | <b>0.595</b> [0.41, 0.74] | <b>0.710</b> [0.60, 0.81] | <b>0.604</b> [0.49, 0.73] | <b>0.730</b> [0.58, 0.86] | <b>0.731</b> [0.57, 0.86] |

The heterogeneous predictor leads the strongest non-learning baseline by  $\Delta\rho = +0.135$  (HGL 0.730 vs Topo-QoS 0.595). Its lead over the *homogeneous* learned baseline is much narrower —  $+0.020$  over GL (0.730 vs 0.710) — and it is not uniform: GL wins on AV and Microservices, HGL wins decisively on IoT (0.891 vs 0.674). In-distribution, therefore, these data do **not** establish that relation-specific message passing is what supplies the learned margin; the two learned families are separated by less than the across-seed spread. The claim that typing matters is carried by the out-of-distribution and in-domain k-fold results below, not by this table, and we state it there rather than here.

**Significance testing, and what it does and does not license.** Table 19 reports the paired Wilcoxon signed-rank test promised in Section 7.3, computed across the seven scenarios on the per-scenario mean  $\rho$ . We report it in full because it qualifies our own headline:

Table 19: Paired Wilcoxon signed-rank tests across the seven scenarios ( $n = 7$ ; two-sided).

| Comparison            | $\Delta\rho$ | Scenarios won | $W$  | $p$   |             |
|-----------------------|--------------|---------------|------|-------|-------------|
| HGL vs Topo-BL        | +0.544       | 7/7           | 0.0  | 0.016 | significant |
| HGL vs GL-QoS         | +0.126       | 6/7           | 5.0  | 0.156 | n.s.        |
| HGL vs Topo-QoS       | +0.135       | 5/7           | 8.0  | 0.375 | n.s.        |
| GL vs Topo-QoS        | +0.116       | 5/7           | 5.0  | 0.156 | n.s.        |
| HGL vs GL             | +0.020       | 5/7           | 11.0 | 0.688 | n.s.        |
| <i>HGL-QoS</i> vs HGL | +0.001       | 3/7           | 13.0 | 0.938 | n.s.        |

Only the comparison against the unweighted structural baseline reaches significance. **The in-distribution**

margin over Topo-QoS — the +0.135 quoted above — is not established by a paired test across scenarios ( $p = 0.375$ ), and neither is the +0.020 over the homogeneous learned model, which the preceding paragraph already declined to claim. Two readings are needed together. The test is genuinely underpowered: at  $n = 7$  paired scenarios the smallest attainable two-sided  $p$  is 0.016, so *only* a clean 7-of-7 sweep can ever reach  $p < 0.05$ , and “not significant” here is not evidence of no effect. But it is equally not evidence of an effect, and the effect is carried by 5 of 7 scenarios with Topo-QoS winning on none of the two it loses by much. We therefore state the in-distribution ranking result as a point-estimate lead that this design cannot confirm, and we do not treat it as the paper’s load-bearing evidence for learning. That role belongs to the critical-set result below, and — with the qualification stated there — to the transfer protocols.

**On the interpretable predictor’s absence from Table 18.**  $Q(v)$  appears in the LOSO table below but not in Table 18. The in-distribution harness scores it under a separate path that does not emit the held-out per-scenario correlations the other six variants produce, so adding a column here would mean quoting a figure computed under a different contract — exactly the defect Section 7.3 documents and corrects. We leave the cell empty rather than fill it inconsistently; Section 8.3’s normalisation and shrinkage sweeps characterise  $Q(v)$ ’s in-distribution ranking behaviour directly, and its inductive result is in Table 20.

Two boundary conditions frame the whole table. The ground truth agrees with *itself* at test–retest  $\rho$  of 0.807–1.000 (Section 7.5), so HGL’s mean 0.730 sits within the spread of what it is scored against rather than underperforming a distant optimum — though the lowest-ceiling scenario is also Microservices (Section 7.5), so the two boundary conditions below are not independent. And Microservices — by construction the sparse, low-centralisation topology with few genuine bottlenecks — is where every learned predictor is weakest (HGL 0.362), while the structural baselines degenerate on Healthcare and IoT ( $\rho \leq 0$ ). No predictor in this study is uniformly best.

**Out of distribution, the typed model leads.** Under Leave-One-Scenario-Out evaluation — the true pre-deployment condition, in which the model must rank a system whose cascade dynamics it has never seen — we obtain:

Table 20: Inductive Leave-One-Scenario-Out evaluation. Cross-fold mean  $\rho$  against  $I^*(v)$ , across-fold standard deviation, and  $F_1@K$  on the held-out scenario.

| Variant          | Mean $\rho$ (LOSO) | Std $\rho$ | $F_1@K$ | Training required |
|------------------|--------------------|------------|---------|-------------------|
| Topo-BL          | 0.105              | 0.151      | 0.179   | no                |
| Topo-QoS         | 0.521              | 0.305      | 0.308   | <b>no</b>         |
| RMAV / $Q(v)$    | -0.093             | 0.140      | 0.209   | no                |
| GL (homogeneous) | 0.436              | 0.120      | 0.440   | yes               |

| Variant                      | Mean $\rho$ (LOSO) | Std $\rho$ | $F_1@K$      | Training required |
|------------------------------|--------------------|------------|--------------|-------------------|
| GL-QoS (homogeneous)         | 0.430              | 0.125      | 0.435        | yes               |
| <b>HGL (typed)</b>           | <b>0.608</b>       | 0.177      | <b>0.465</b> | yes               |
| <i>HGL-QoS</i> (typed + QoS) | 0.595              | 0.190      | 0.461        | yes               |

Topo-QoS is a QoS-weighted centrality that requires no training, no labels, and no transfer assumption; because it is never fitted, its out-of-distribution score *is* its score. HGL reaches  $\rho = 0.608$  against its 0.521, and leads on  $F_1@K$  as well (0.465 vs 0.308). The same ordering holds under the in-domain k-fold protocol, where the separation is wider still — HGL-QoS 0.693 and HGL 0.666 against Topo-QoS 0.492 — and where the typed models are also the *most stable* across folds ( $\sigma = 0.07$  against 0.34 for Topo-QoS).

**This comparison is only meaningful because the baseline was repaired first.** In an earlier revision, Topo-QoS scored *no* QoS weighting at all on the logical-dependency substrate: QoS is declared on the Topic node, but the harness looked for it on the pub-sub relationship, which the generated topologies emit without one. The lookup never matched, every derived dependency edge kept a unit weight, and the QoS-weighted baseline silently computed plain betweenness on all seven scenarios — it was Topo-BL under another name. We resolve  $w(t)$  from the shared Topic instead, taking the strongest contract when a pair communicates over several topics. A baseline that is accidentally identical to the one it is meant to improve on will always flatter whatever it is compared against, and the figures above are reported only after that defect was removed.

Even so, we state the ranking result carefully, because of this paper’s own history with it. An earlier version reported typed learning as the out-of-distribution winner on figures produced by an untrained sweep (Section 9.2); a later version, after correcting the evaluation contract, reported a tie and stated plainly that learning did not beat the training-free baseline on rank correlation. On the regenerated corpus, rebuilt ground-truth caches (Section 7.1) and repaired baseline, the ordering favours the typed model. A conclusion that has moved this often under changes to the *measurement apparatus* rather than to the method deserves an explicit statement of what would settle it: an evaluation on topologies that do not share a generator (Section 9.3), which this submission does not have.

We must add a second, sharper qualification, which we prefer to state than to let a reader discover. **Unlike Table 18, the run behind Table 20 was not persisted to a retained artifact.** Table 18 regenerates exactly from a stored result file; the corresponding Leave-One-Scenario-Out result file was overwritten during the revision, and the most recent retained LOSO run log — produced *before* the baseline repair described above — records a different ordering, with Topo-QoS at  $\rho = 0.609$  against HGL at 0.597 and  $F_1@K$  of 0.416 against 0.548. That is the ordering under which the training-free baseline ties the typed model on rank correlation. We report the post-repair figures in Table 20 because they come from the corrected apparatus,

but three changes are confounded in the interval between the two runs — the baseline repair, the corpus regeneration, and the ground-truth cache rebuild — and we cannot attribute the shift to the repair alone. Readers should therefore treat Table 20’s *ranking* margin as provisional pending a re-run under the final apparatus with the artifact retained, and weight the  $F_1@K$  column, which favours the typed model under both runs, accordingly. This is the same class of defect Section 9.2 discloses, caught one revision later, and we regard the standing lesson as the one stated there: a result whose artifact is not retained is not yet a result.

**The learned advantage is most robust in set identification.**  $F_1@K$  — the overlap between the predicted and actual top- $K$  critical sets — favours the typed model by a wider relative margin than  $\rho$  does: 0.465 for HGL against 0.308 for Topo-QoS, a 51% relative improvement, with the same ordering holding for both homogeneous learned baselines over both structural ones. This matters operationally more than the ranking result. An architect does not consume a total order over 150 components; they consume a shortlist. Unlike the  $\rho$  comparison, this one does not invert under any of the three protocols we ran.

RQ1 therefore resolves as a scope condition rather than a verdict. Learning leads on the point estimate under all three protocols, on both metrics — but the *ranking* half of that answer is not established: it fails the paired significance test in-distribution ( $p = 0.375$ , Table 19), it rests on an unretained artifact out of distribution, and the strongest non-learning comparator is degraded on 2 of 7 scenarios. The set-identification advantage is the more defensible half, being the one result that survives both the protocol changes and the apparatus changes documented above. Three caveats bound even that: the LOSO across-fold standard deviation remains substantial (0.177 for HGL), top- $K$  metrics inherit the label churn documented in Section 7.5 — the ground truth’s own top- $K$  set agrees with itself at Jaccard 0.44–1.00 across seeds, with Microservices the worst at 0.44 — and the interpretable RMAV predictor  $Q(v)$  does not transfer at all ( $\rho = -0.093$  LOSO,  $-0.123$  k-fold), which is a negative result about the composite score’s ranking use, not about its attribution use (Section 8.3). The practical recommendation we are willing to defend is correspondingly narrow: use typed learning when the deliverable is a shortlist, and treat the ranking comparison as open.

## 8.2. RQ2 — What Taking Node and Edge Type Seriously Shows (and Does Not Show)

We report four analyses that take node and edge *type* seriously: one positive result, one newly measured result, and two negative ones.

**Heterogeneity is the dominant source of predictive gain — but only where the model must generalise.** Isolating architecture from QoS encoding, the typed model’s advantage over the homogeneous baseline is negligible in-distribution ( $\Delta\rho = +0.020$ ; HGL 0.730 vs GL 0.710) and grows sharply as the evaluation moves away from the training distribution:  $+0.172$  under LOSO (0.608 vs 0.436) and  $+0.257$  under in-domain k-fold (0.666 vs 0.409). The typed model is also far more stable across folds ( $\sigma = 0.07$  vs 0.15 for GL under k-fold).

That pattern is the interesting form of the result, and it is not the one we previously reported. When train and test come from the same topology, a homogeneous GAT can recover most of what typing provides, because the type signal is largely redundant with structure it can observe directly. When the model must rank a system whose cascade dynamics it has never seen, the relation-specific message passing is what carries over. Collapsing pub-sub types into a single node class discards information that survives the distribution shift; the ablation isolates that as the source of the gain, and locates it in generalisation rather than in fit. On  $F_1@K$  the two learned families are much closer out of distribution (0.465 vs 0.440), so this is a claim about ranking transfer specifically, not about every metric.

**Edge criticality, now measured rather than inferred.** Earlier versions of this framework labelled edges by projecting node labels through a hand-chosen bridge multiplier,  $I_{\text{edge}}(u, v) = I^*(u) \times \{1.0 \mid 0.1\}$ . That is an assumption about edge importance, not an observation of it. We now remove each candidate relationship — leaving both endpoints active, which is precisely the partial-outage case that distinguishes edge from node criticality (Section 4.1) — and recompute impact against a no-op control. The control subtraction is load-bearing: the impact function is non-zero on an untouched graph, because topics that already lack a publisher or subscriber count as lost throughput, so a level rather than a delta would hand every edge that floor as apparent signal.

The candidate set on `av_system` — structural bridges united with the highest-betweenness structural edges — contains 50 edges (35 `RUNS_ON`, 11 `CONNECTS_TO`, 3 `SUBSCRIBES_TO`, 1 `PUBLISHES_TO`), of which exactly 4 carry non-zero impact: the one `PUBLISHES_TO` edge and all three `SUBSCRIBES_TO` edges, each connecting a shared library to the topic it produces or consumes. Two findings follow. First, **most individual links are replaceable, in both magnitude and count**: the largest measured impact of severing any single relationship is 0.00504, over an order of magnitude below the largest single-*component* impact in the suite ( $I_{\text{comp}} = 0.320$ , Section 5.4), and 46 of 50 candidates measure exactly zero. That is the substantive answer to the replaceability question Section 4.7 poses, and it is not what the bridge heuristic implied — the heuristic would have assigned a bridge edge its source node’s full blast radius. Second, the measurement confirms a modelling gap the heuristic concealed: every one of the 46 `RUNS_ON/CONNECTS_TO` candidates — the majority of the set, and structurally non-redundant bridges by construction — scores exactly zero, because the cascade routes no traffic over infrastructure-layer relations at all. Bridge detection flags these links as non-redundant; the measurement means *this model cannot express that link’s failure, not that link does not matter* — the same caveat that applies to Topic and Node labels (Section 7.5).

**This measurement does not validate the attribution of Section 4.7, and is not intended to.** The sweep severs raw structural edges, whereas  $Q(u, v)$  is defined on derived `DEPENDS_ON` edges; the two are computed over populations that barely intersect by construction (Section 4.7), so no correlation between them is reported here and none should be inferred from the two results appearing in the same section.

**A methodological note on reproducing this figure.** The candidate set above requires the sweep to

be run against a freshly loaded repository, before any structural analysis has touched it. We found during this revision that running the analysis and prediction stages against the same in-memory repository instance *before* constructing the simulator’s graph view causes derived ROUTES and DEPENDS\_ON edges to leak into what the simulator receives as  $G_{\text{structural}}$  — a repository state-ordering issue distinct from, and not caught by, the import-level independence check of Section 5.3. It is not visible in this paper’s other results, since the standard pipeline order is Simulate before Predict throughout, but it can silently substitute a contaminated candidate set for a clean one in an ad hoc script, which is precisely how an earlier revision of this figure was produced. We flag it as a reproducibility hazard for this specific measurement rather than treat it as resolved.

**The shared-library blast mechanism: tested, and not confirmed as a low- $Q$ /high- $I$  gap.** Shared libraries have a structurally distinct simultaneous-failure mode (Section 3.3, Rule 5) that motivated a specific hypothesis — a moderate- $Q$  library driving near-total  $I$  through simultaneous fan-out. We tested this directly across all seven scenarios (165 Library nodes) and did not find it (Section 5.4): the highest library  $Q$  in the suite is 0.422 with  $I_{\text{comp}} = 0.086$ ; no library has  $I_{\text{comp}}(v)$  exceeding  $Q(v)$ ; and the largest single-component impact of any type is  $I_{\text{comp}} = 0.320$ , well short of near-total. We report this as a negative result rather than adjust the hypothesis. The simultaneous-blast *mechanism* remains real and worth modelling; this suite does not exhibit the mismatch it was expected to expose.

**A scoping caveat this analysis must carry.** The library and stratified analyses in Section 5.4 and Section 5.5 are computed against  $I_{\text{comp}}(v)$ , whereas Tables 18 and 20 are computed against  $I^*(v)$ . Those two oracles agree at mean  $\rho = 0.394$  (Section 7.5). The negative library result is therefore a statement about  $I_{\text{comp}}$ , and does not license a corresponding claim about the  $I^*$ -backed tables. We flag this rather than let adjacency in the text imply mutual support.

### 8.3. RQ3 and Robustness — Ablations and Sensitivity

**QoS encoding (RQ3): a null result in all three regimes.** Adding explicit QoS edge attributes to the typed model moves accuracy by less than the across-seed spread, and the *sign of the effect depends on the protocol*: in-distribution  $\rho = 0.731$  for *HGL-QoS* against 0.730 for HGL (+0.001), out of distribution 0.595 against 0.608 (−0.013), and under in-domain k-fold 0.693 against 0.666 (+0.027). An effect that changes direction across evaluation protocols while remaining an order of magnitude smaller than the fold-to-fold variance is a null, and a cleaner one than the single-regime comparison previously reported. An earlier version of this paper reported QoS encoding as the primary driver of the out-of-distribution gain ( $\rho = 0.401$  vs 0.307); those figures came from the untrained sweep of Section 9.2 and do not survive re-measurement. The plausible reading is the one the in-distribution result already suggested: the lifted dependency topology encodes most QoS-relevant routing, so the extra dimensions mainly enlarge the parameter space. RQ3 resolves as a null result in every regime we measured, which we report as stated.

1303 **Dimension-weight sensitivity: no plateau, and equal weights win.** Sweeping the shrinkage  
1304 parameter  $\lambda$ , which blends the stated dimension weighting toward a uniform prior ( $\lambda = 0$  is equal weights,  
1305  $\lambda = 1$  the raw judgement). The QoS-profile adaptation of Section 4.3 remains active throughout the sweep,  
1306 as it is in every run reported in this paper: each  $\lambda$  therefore fixes the vector that adaptation starts from, not  
1307 the coefficients any individual scenario is finally scored with. The sweep is consequently a sensitivity analysis  
1308 of the *stated ordering* under the framework's normal operating configuration, and the  $\lambda$  labels should be read  
1309 as inputs to the weighting path rather than as the applied weights:

Table 21: AHP shrinkage sensitivity. Mean  $\rho$  against  $I^*(v)$  as  $\lambda$  blends the stated weighting toward a uniform prior.

| $\lambda$   | 0.00         | 0.50  | 0.60  | 0.65  | <b>0.70</b>  | 0.75  | 0.80  | 0.90  | 1.00  |
|-------------|--------------|-------|-------|-------|--------------|-------|-------|-------|-------|
| mean $\rho$ | <b>0.292</b> | 0.206 | 0.191 | 0.187 | <b>0.181</b> | 0.174 | 0.167 | 0.152 | 0.140 |

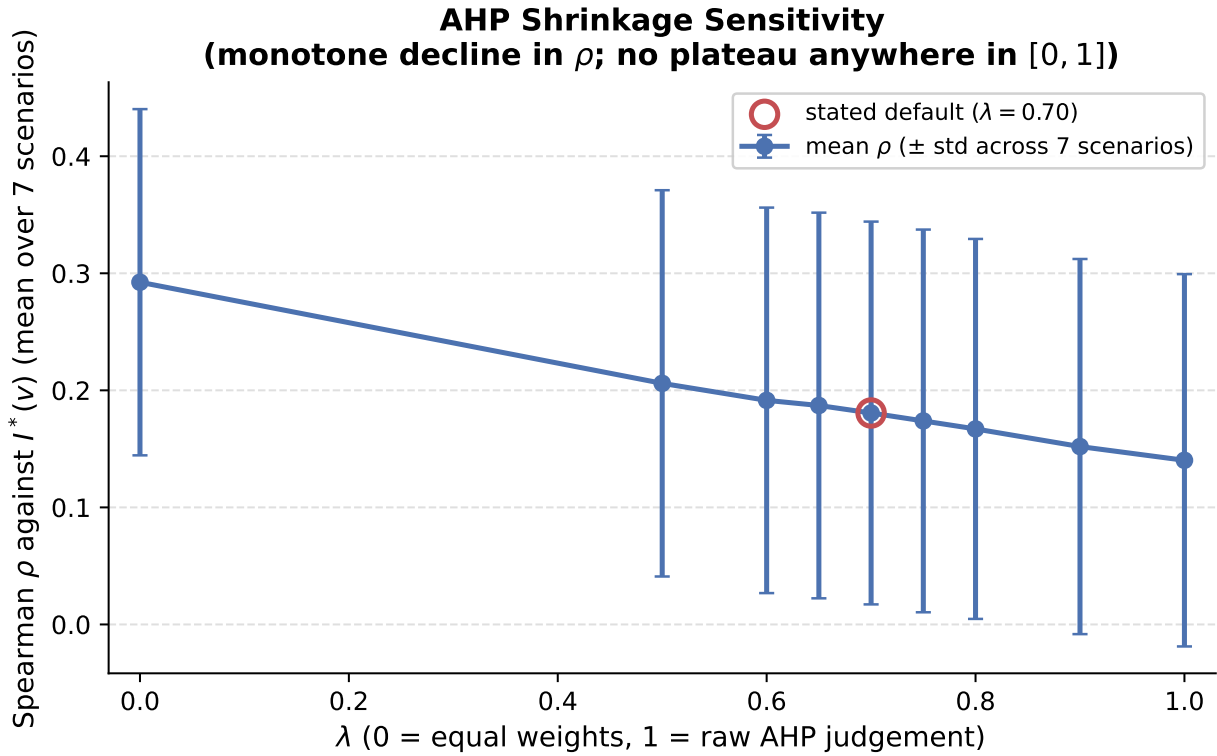


Figure 5: mean  $\rho$  against  $\lambda$  over the shrinkage sweep, showing the monotone decline and the absence of a plateau.

1310  $\rho$  is monotonically decreasing in  $\lambda$ . There is no plateau anywhere in the range, and equal dimension  
1311 weights outperform the calibrated  $\lambda = 0.70$  setting by 0.111. An earlier version of this paper claimed a

plateau over  $\lambda \in [0.65, 0.75]$ ; that claim was not backed by a committed artifact and is contradicted by this sweep. The sweep has since been re-run against the regenerated corpus and rebuilt caches (Section 7.1) and the conclusion is unchanged in direction and magnitude — this is the one robustness result in Section 8.3 that did not move under re-measurement.

We draw the corresponding conclusion about the contribution rather than defending the weighting. **The value of the RMAV decomposition is attribution, not ranking accuracy.** A composite score ranks; a four-dimensional profile explains *why* a component ranks where it does, and routes the finding to the engineering role equipped to act on it (Section 4.1) — a structural single point of failure and a cascade hub call for different remediations even at identical composite scores. That explanatory function is unaffected by the weighting result. What the sweep removes is any claim that the specific weights improve predictive accuracy; on this cohort they do not, and a practitioner optimising for ranking alone should use equal weights.

**Normalisation sensitivity.** The default rank-based normalisation discards magnitude before the weighted sum, which makes  $Q(v)$  closer to a Borda count over the structural metrics than to a weighted aggregate. Measured against  $I^*$ : rank (robust)  $\rho = 0.181$ , min-max 0.318, z-score 0.318. Retaining magnitude is worth  $\approx +0.137 \rho$ . The outlier-robustness argument for rank normalisation is real but is outweighed here; we retain the default so that previously reported figures remain interpretable, and report the sweep alongside.

**Propagation-threshold sensitivity.** Because the ground truth itself depends on `propagation_threshold`, we report  $\rho$  across its range rather than at a single value:

Table 22: Propagation-threshold sensitivity. Mean  $\rho$  against  $I^*(v)$  across the sweep; the canonical default is 0.20.

| threshold   | 0.00  | 0.10  | <b>0.20</b>  | 0.35  | 0.50  | 0.75  | 1.00  |
|-------------|-------|-------|--------------|-------|-------|-------|-------|
| mean $\rho$ | 0.001 | 0.109 | <b>0.194</b> | 0.227 | 0.226 | 0.230 | 0.231 |

The conclusions *do* depend on this parameter:  $\rho$  spans 0.230 across the sweep, the canonical 0.2 default sits below the plateau the curve reaches from 0.35 upward, and at 0.0 — where any feed loss triggers a cascade — the correlation vanishes entirely. We therefore do not claim threshold-independence. The direction is interpretable: a higher threshold admits only components whose failure genuinely starves their dependents, which is closer to what the structural score is built to detect. Remediation edits (Section 6.4) are required to improve impact across the entire sweep precisely because a single-threshold result is not trustworthy here.

#### 8.4. RQ4 — Feasibility and Performance of SaG as a CI/CD Quality Gate

A primary blocker for continuous Static System Analysis (SSA) is execution time: developers will bypass or disable quality gates that introduce significant build delays. We evaluate the feasibility of deploying SaG



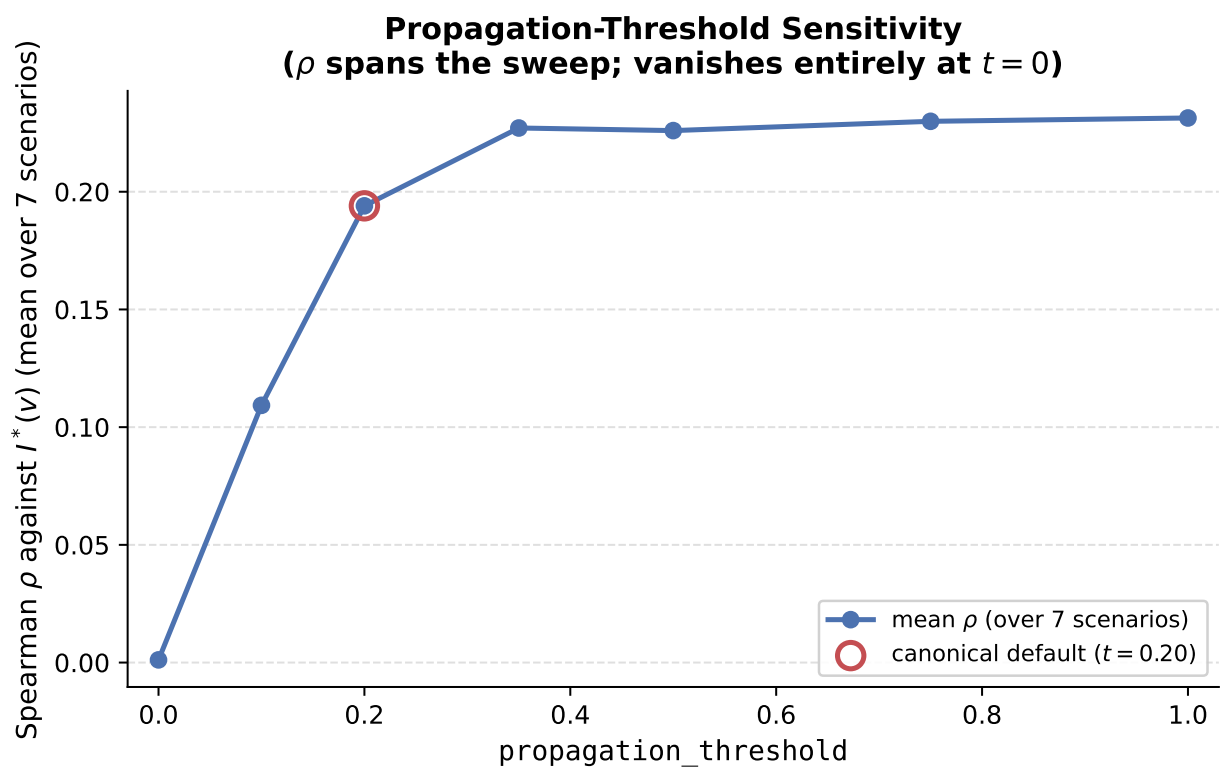


Figure 6: mean  $\rho$  against `propagation_threshold`, with the canonical 0.2 default marked.

as a blocking gate by measuring the wall-clock cost of the structural analysis and anti-pattern catalog — the mechanism `detect_antipatterns.py` invokes — run via the in-memory `MemoryRepository`, across all eleven scenarios in our corpus (mean over the five canonical seeds).

The measured footprint: -  $\leq 90$  components (`tiny_system` and all three real-world transcribed architectures): 0.02–0.04 s. - **98–326 components** (the remaining six generated scenarios): 0.27–1.24 s. Cost does not scale monotonically with component count in this range — `hub_and_spoke_system` (139 components, 1.24 s) costs more than `iot_smart_city_system` (326 components, 1.08 s) — consistent with the catalog’s cost being driven by specific detectors’ complexity (e.g. `DEEP_PIPELINE`’s path enumeration) more than by raw component count. - **enterprise\_system**, the largest scenario at 520 components:  $26.74 \pm 0.32$  s.

All eleven scenarios complete in well under the several-minute budget continuous build pipelines typically allow.

In terms of gating efficacy: the gate is currently absolute rather than delta-aware (Section 6.6), so there is no merge-base diff to evaluate detection against; what we can and do measure is the anti-pattern catalog’s raw agreement with the cascade oracle — the property the gate is a proxy for. Scoring CRITICAL/HIGH findings (with edge-keyed findings crediting both endpoints) against the oracle’s own critical set gives, across five seeds: precision  $0.237 \pm 0.014$ , recall  $0.887 \pm 0.059$ , F1  $0.374 \pm 0.022$ , Cohen’s  $\kappa = -0.036 \pm 0.049$  on the seven generated scenarios ( $n = 40$  scenario–seed pairs), and precision  $0.402 \pm 0.059$ , recall  $0.861 \pm 0.105$ , F1  $0.544 \pm 0.061$ ,  $\kappa = 0.296 \pm 0.100$  on the three real-world architectures ( $n = 15$ ). Recall is high and precision is not: the catalog over-flags relative to the oracle’s critical set, and near-zero  $\kappa$  on the generated corpus means the agreement it does show is close to what chance flagging at the catalog’s own base rate would produce. We read this as a genuine limitation of the pattern catalog as a *stand-alone* predictor of simulated impact — a finding pattern catalog and rank predictor serve different purposes (naming a structural smell versus ranking by predicted impact, Section 9.1), but the gap here is wide enough that the catalog’s findings should not be read as impact-calibrated. The composite  $Q(v)$  of Section 4 remains the ranking signal validated against the oracle in Section 8.1.

From a **sustainability and resource efficiency** standpoint, evaluating architectural risks statically in-memory (0.02 s–26.74 s across our corpus) yields energy savings relative to spinning up staging clusters or running heavier dynamic checks per build, though we have not measured that comparison directly.

## 8.5. RQ5 — Real-World Open-Source System Architecture Validation

To evaluate operational generalizability beyond synthetic topology generation, we evaluate SaG on three authentic real-world open-source software architectures (Section 7.1): 1. **Autoware.universe (ROS 2 Autonomous Driving Platform)**: A real-world cyber-physical software graph comprising 32 Applications, 24 Topics with explicit DDS QoS contracts (`RELIABLE/BEST_EFFORT`, `TRANSIENT_LOCAL/VOLATILE`), 3 Brokers

(CycloneDDS, FastDDS, Zenoh), 6 Deployment Nodes, 10 Shared C++ Libraries (`autoware_universe_utils`, `tier4_autoware_utils`), and realistic SonarQube code quality metrics. **2. Production Cloud-Native Microservices Mesh:** A real-world cloud-native software graph based on the Google Online Boutique benchmark, comprising 22 Microservices, 20 Topics across Kafka, RabbitMQ, Redis PubSub, and NATS, 6 Kubernetes/Cloud nodes, and 8 shared helper libraries. **3. Train-Ticket Railway Booking Mesh:** A real-world cloud-native software graph based on the Fudan University Train-Ticket benchmark, comprising 41 Microservices, 30 Topics across RabbitMQ and Redis PubSub with Spring Eureka service discovery, 8 deployment Nodes, and 8 shared Spring/MyBatis libraries — at 90 components, the largest of the three.

All three rows are produced by the framework’s validation sweep over five seeds ( $\{42, 123, 456, 789, 2024\}$ , matching Section 7.4), with no QoS enrichment, against the component-level cascade oracle of Section 5.1: Spearman  $\rho$  and Kendall  $\tau$  are computed over the Application population (the other four node types carry constant or near-constant simulated impact on these graphs and contribute no rank information, per the same coverage limitation as the synthetic suite, Section 7.5);  $K$  is  $\lceil 0.20 \times |V| \rceil$  applied within that population (15 of 32 Applications for Autoware, 12 of 22 for Cloud Microservices, 18 of 41 for Train-Ticket); and all three scenarios are classified **sparse** by the tool’s topology-class rule, which sets the gate thresholds below. Reported  $\rho$  is the seed mean  $\pm$  standard deviation, not a single-seed point estimate: **FaultInjector** tie-breaks intra-wave propagation stochastically by design (Section 5.1), so — unlike the deterministic RMAV/ $Q(v)$  scores — the simulated labels, and therefore  $\rho$ , genuinely vary *across* the five seeds within one sweep. At a *fixed* seed the label is now reproducible process to process; an earlier draft of this evaluation was not, for the instrument-defect reason disclosed in Section 9.2, and the figures below are the corrected, reproducible ones. The *ranking* is nonetheless stable across seeds (Rank Consistency Rate = 1.000 for all three), which is why  $F_1@K$  does not carry the same  $\pm$  as  $\rho$  below. All three runs are reproducible on demand from the replication package; see the data-availability statement for what is archived.

Table 23: Real-world open-source architecture validation, five seeds, against the component-level cascade oracle of Section 5.1.

| Real-World Architecture                | Nodes | Apps | Spearman $\rho$ (mean $\pm$ std)    | Kendall $\tau$ (mean) | $F_1@K$      | Tie-robust $F_1@K$ | Non-zero $I$ | Predictive Gain (vs DC) | SPOF-F1 | Gate        |
|----------------------------------------|-------|------|-------------------------------------|-----------------------|--------------|--------------------|--------------|-------------------------|---------|-------------|
| <b>Autoware.universe 75 (ROS 2)</b>    |       | 32   | <b>0.688 <math>\pm</math> 0.009</b> | 0.517                 | <b>0.800</b> | 0.800              | 19/32        | +0.360                  | 0.500   | <b>FAIL</b> |
| <b>Cloud-Native Microservices Mesh</b> | 60    | 22   | <b>0.778 <math>\pm</math> 0.001</b> | 0.639                 | <b>1.000</b> | 0.760              | 8/22         | +0.014                  | 0.333   | <b>FAIL</b> |

| Real-World Architecture | Nodes | Apps | Spearman $\rho$ (mean $\pm$ std)    | Kendall $\tau$ (mean) | $F_1@K$      | Tie-robust $F_1@K$ | Non-zero $I$ | Predictive Gain (vs DC) | SPOF-F1 | Gate        |
|-------------------------|-------|------|-------------------------------------|-----------------------|--------------|--------------------|--------------|-------------------------|---------|-------------|
| <b>Train-Ticket</b>     | 90    | 41   | <b>0.759 <math>\pm</math> 0.001</b> | 0.605                 | <b>1.000</b> | 0.810              | 14/41        | +0.264                  | 0.571   | <b>FAIL</b> |
| <b>Railway</b>          |       |      |                                     |                       |              |                    |              |                         |         |             |
| <b>Booking</b>          |       |      |                                     |                       |              |                    |              |                         |         |             |
| <b>Mesh</b>             |       |      |                                     |                       |              |                    |              |                         |         |             |

**Key Findings:** 1. Rank correlation is strong on two of three architectures and closest to the framework’s own gate on Train-Ticket, but all three fail it overall. Cloud Microservices ( $\rho = 0.778 \pm 0.001$ ) and Train-Ticket ( $\rho = 0.759 \pm 0.001$ ) clear the  $\rho \geq 0.75$  gate threshold; Autoware does not ( $0.688 \pm 0.009$ , short by 0.062) and carries the most seed-to-seed variance of the three — roughly seven to fifteen times larger than Train-Ticket’s  $\sigma$  and Cloud Microservices’. An earlier draft of this sweep additionally reported the *sweep-to-sweep* mean and standard deviation themselves as unstable at a fixed seed set (mean 0.6956–0.6959, std 0.011–0.015 across repeated runs), which we took at the time to be a further, second-order property of this graph. It was not: the cause was the instrument defect disclosed in Section 9.2 (an unordered iteration inside **FaultInjector** fed seeded random draws in a process-dependent order), now fixed. Sweep-to-sweep reruns at the figures above are identical to the precision shown. All three nonetheless fail the **sparse-topology** gate as a whole, on **SPOF-F1**  $\geq 0.6$ : Autoware 0.500, Cloud Microservices 0.333, Train-Ticket 0.571 — closest of the three, short by only 0.029 — and SPOF-F1 is exactly stable across seeds for all three (it depends on the deterministic articulation-point flag and a fixed 0.3 impact threshold). Cloud Microservices additionally fails predictive gain  $\geq 0.02$  (0.014). SPOF-F1 scores agreement between structural articulation points and components whose simulated impact exceeds 0.3; on graphs this size a handful of disagreements move the F1 sharply, and that is not more forgiving on hand-transcribed real-world graphs than on the synthetic suite. We report the failing gate rather than the passing correlations alone, because presenting one without the other would overstate what these three cases establish. 2. **Set-containment of the genuinely critical components is real; the reported  $F_1@K = 1.000$  on two of three graphs is partly a tie-breaking artifact, and we report both.** On Cloud Microservices and Train-Ticket,  $K$  (12 and 18) exceeds the number of Applications carrying non-zero simulated impact (8 and 14 respectively), so the “actual top- $K$ ” set is padded with components tied at  $I = 0$ . Because both the predicted and actual orderings are produced by the same stable sort, the tie-padding lands on the *same* arbitrary components in both, which is what drives  $F_1@K$  to a perfect 1.000 on both graphs. Re-sorting under 200 random shuffles of the tied region gives a tie-robust  $F_1@K$  of 0.760 (Cloud Microservices) and 0.810 (Train-Ticket); Autoware is unaffected (19 non-zero exceeds  $K = 15$ , so no boundary tie exists, and both figures agree at 0.800). The genuine, tie-independent finding is **set containment**: every one of the 8 non-

zero-impact Cloud Microservices applications and every one of the 14 non-zero-impact Train-Ticket services falls somewhere inside the respective predicted top- $K$  — a real result, distinct from the exact top- $K$  ordering claim that  $F_1@K$  makes. On Train-Ticket, the highest-impact services recovered are `ts-ui-dashboard` ( $I = 0.545$ ), `ts-auth-service` ( $I = 0.397$ ), `ts-gateway-service` ( $I = 0.384$ ), `ts-security-service` ( $I = 0.370$ ) and `ts-user-service` ( $I = 0.366$ ); on Cloud Microservices, `checkout-service`, `payment-service`, `fraud-detection-service` and `order-processor-service` are among the 8 recovered. On Autoware, 12 of the predicted top-15 fall in the actual top-15 ( $F_1@K = 0.800$ , no tie artifact), correctly including the perception/localization hubs `lidar_centerpoint_node`, `ndt_scan_matcher`, `multi_object_tracker`, `ekf_localizer` and `velodyne_node_container`. The three misses are worth naming rather than glossing over: `vehicle_cmd_gate` is a **false positive** —  $Q(v)$  ranks it 6th by structural score, but its simulated impact is 0, since the cascade oracle attaches no downstream loss to this particular actuator on this topology. `obstacle_avoidance_planner` and `behavior_velocity_planner` are the same pattern. We checked whether the same padding could inflate the synthetic-suite  $F_1@K$  figures of Section 8.1 (Table 18, the LOSO table) and found it does not: the smallest margin between  $K$  and the non-zero-impact Application count across the seven scenarios is `av_system`’s  $K = 16$  against 43 non-zero components, so the boundary is never reached there. This is the real-world instance of the general caveat Section 7.5 states for the synthetic suite: a structural score can be confidently wrong about a component the cascade model does not route traffic through in a way that registers impact, and a safety-relevant name in the predicted set is not evidence the prediction is correct.

**3. Predictive gain over degree centrality is real but small and graph-dependent, and fails its own threshold on one of three graphs.** SaG’s  $|\rho|$  exceeds degree centrality’s  $|\rho|$  against the same labels by +0.361 on Autoware, +0.264 on Train-Ticket, and +0.014 on Cloud Microservices — the last below the 0.02 gate threshold, meaning typed dependency semantics add essentially nothing over raw degree on that particular graph. With a third point, the pattern from the two-graph comparison holds rather than looking like an artifact of one outlier: the margin over an untyped baseline is graph-dependent, not a fixed advantage, consistent with the scenario-to-scenario variation already observed on the synthetic suite (Section 8.1).

**What these three cases do and do not establish.** Four scoping conditions apply, and they matter because this is the paper’s only evidence outside the generator.

*They are hand-built models of real architectures, not harvested artifacts.* Each graph was constructed by transcribing a published system’s component inventory, topic set and declared QoS into the schema of Section 3.1. What transfers is therefore the *topology and QoS structure* of a real system, not its runtime behaviour. A harvested graph — extracted automatically from a running deployment or a build system — would be stronger evidence and is not what we have.

*The ground truth is still simulated.* These correlations are between a structural score and a simulated impact label, produced by the same machinery as everywhere else in this paper. No incident record, operator

judgement, or observed failure enters. Section 9.2’s construct-validity bound applies here unchanged: this is agreement with a model of harm, not with harm.

*They are small, and D4 forbids comparing them.* At 75, 60 and 90 components these graphs are smaller than five of the seven synthetic scenarios, and because criticality is relative to a system’s own distribution (D4), the three  $\rho$  values are separate within-system results, not three points on a shared scale — the gaps between them are not a finding about the three domains.

*They are three systems from two paradigms.* Cloud Microservices and Train-Ticket are both microservice meshes; only Autoware represents a distinct cyber-physical paradigm. Three points still cannot establish a generalisation over production software, and the paradigm count is two, not three, which is a weaker diversity claim than three independent architectural styles. What the three cases do establish is the narrower and still useful claim that the framework runs end-to-end on externally-specified architectures and recovers a ranking there at least as well as on generated ones, on two distinct meshes independently rather than on one — evidence against the concern that its performance depends on regularities of our own generator (Section 9.3), without settling it.

---

## 9. Discussion, Threats to Validity, and Conclusion

### 9.1. Interpretation

The results converge on a single message: for pre-deployment criticality analysis of pub-sub middleware, *how* a component is critical is at least as important as *whether* it is — and the case for graph learning is narrower, and more specific, than we expected when we set out. Four findings carry this.

**First, learning pays — most defensibly for set identification (RQ1).** The typed predictor leads the strongest non-learning baseline on both metrics and under all three evaluation protocols. Out of distribution — the genuine pre-deployment condition — it reaches  $\rho = 0.608$  against 0.521 for a QoS-weighted centrality that requires no training, no labels and no transfer assumption, and  $F_1@K = 0.465$  against 0.308. Both margins are measured against a baseline we first had to repair: **Topo-QoS** was computing no QoS weighting whatsoever (Section 8.1), and until that was fixed it was **Topo-BL** wearing a different label. We place substantially more weight on the second metric than the first, for three reasons. Operationally, an architect hardens a handful of components, not a ranked list of 150. Empirically, the ranking comparison is the less stable one: **Topo-QoS** carries the largest across-fold variance of any predictor in the study ( $\sigma = 0.31$  LOSO, 0.34 k-fold), whereas the set-identification ordering does not invert under any protocol. And evidentially, the ranking margin is the weaker of the two: it does not survive a paired test across scenarios in-distribution ( $p = 0.375$ , Table 19), and out of distribution it rests on a run whose artifact was not retained, against an earlier retained run that recorded a tie (Section 8.1). The practical reading is therefore a scope condition

with an asymmetry in confidence: if a team needs the critical set, typed learning is worth its training cost on the evidence here; if they need a cheap ordering, QoS-weighted centrality remains a serviceable default, and we cannot presently demonstrate that learning beats it.

**Second, decomposition is worth having for reasons that are not accuracy (RQ2, Section 8.2; weighting sensitivity from RQ3’s robustness analysis, Section 8.3).** The dimension weighting does not improve ranking — equal weights beat the calibrated ones (Section 8.3) — and the stratified check we ran to detect Simpson’s-paradox masking did not find it in the  $Q$ – $I$  relation. What survives is the property we think actually motivates the decomposition: a four-dimensional profile says *why* a component is critical and routes the finding to an owner, which a scalar cannot, and that holds regardless of how the four are combined. The methodological discipline was also not wasted: pooled-versus-stratified reporting *did* catch a real distortion elsewhere in this study, where collapsing Application and Library nodes into one correlation moved a headline figure by 0.38 (Section 5.5). The check earned its place by catching something, just not where we pointed it.

**Third, edge criticality benefits from being measured rather than assumed.** Replacing a hand-chosen bridge multiplier with actual edge-removal simulation reversed the intuition it encoded: most individual links turn out to be replaceable, and a whole class of structurally non-redundant edges (`RUNS_ON`) carries no measurable impact at all because the cascade model cannot express their failure. The heuristic would have assigned exactly those edges their source node’s full blast radius. This is a small result with a general moral — a plausible label-generating assumption is not a substitute for the observation it stands in for.

**Fourth, remediation is now verified per edit, which is a stronger guarantee than the previous aggregate but not yet a demonstration of value (Section 6.7).** Each candidate is simulated in isolation and admitted only if it beats the simulator’s own noise at every propagation threshold, so a regressing edit can no longer be carried by an improving one. What that filter reveals is that the operators’ yield is highly topology-dependent — from 3 of 35 candidates on Autonomous Vehicle to 38 of 58 on IoT Smart City — and that the resulting risk reductions are uniformly small. We regard this as the correct outcome of an honest test rather than a failure of the mechanism: the previous design reported a more favourable aggregate precisely because it never asked each edit to justify itself.

**Finally, automated quality gating operationalises these checks continuously (RQ4).** By evaluating in-memory via the `MemoryRepository` and bypassing database round-trips, the framework runs anti-pattern scans and counterfactual simulations in seconds (~5 s medium, ~40 s xlarge). That speed makes the analyzer viable as a blocking CI/CD check, and the delta-aware gate semantics (Section 6.6) make it sustainable: it blocks newly introduced architectural regressions at commit time — bridging the Architecture–Code Gap — without repeatedly flagging known, risk-accepted structure, in the manner of “Clean as You Code” static-analysis gates. Of the four contributions this is the one least disturbed by the

audit, and the one we would defend most confidently.

## 9.2. Threats to Validity

**Construct validity.** D1 and D2 define criticality as Quality-in-Use loss, and this study never observes Quality-in-Use. The validation chain has two links, and only the first is measured:

$$\underbrace{\text{structural / learned score}}_{Q(v), \text{ HGL}} \xrightarrow{\textcircled{1}} \underbrace{\text{simulated failure impact}}_{I^*, I_{\text{comp}}, I_{\text{dyn}}} \xrightarrow{\textcircled{2}} \underbrace{\text{real Quality-in-Use loss}}_{\text{what D1 and D2 define}} \quad (25)$$

Link ① is what Section 8 reports: a real, falsifiable result. Link ② is not measured anywhere in this paper — no user study, expert elicitation, or production incident record is used, and the simulator is itself a *model* of stakeholder harm rather than an observation of it. The defensible claim is therefore: *RMAV and the learned predictors track simulated failure impact, and simulated failure impact is our stated operationalisation of Quality-in-Use loss.* The stronger claim — that these scores track Quality-in-Use as stakeholders would report it — is not supported by anything here, and we do not make it. Closing link ② requires evidence of a different kind: expert ranking studies on the same topologies, or post-hoc comparison against incident records from a deployed system (Section 9.3).

Two qualifications keep this from being either overstated or unduly bleak. The ISO/IEC 25019 characteristics are not equally out of reach: Effectiveness and Efficiency are in principle measurable from quantities the simulators already produce — delivery rate before and after a fault, and the latency shift the discrete-event engine records — so link ② is partly closable by re-summarising existing output on the Quality-in-Use axis rather than by new instrumentation. Freedom from risk is blocked by the corpus rather than by the method: deadline and lifespan violation counters exist and the harness has an oracle slot for them, but no topic in the scenario corpus declares a deadline, so the counters never fire. Acceptability and Satisfaction are behavioural and are not measurable by these means at all, which bounds what this construct can ever claim on them. None of that has been *run*; it establishes measurability, not a measurement.

Because the ground truth is simulated rather than observed, the strongest claims we can make remain comparative: which modelling choices perform better under identical conditions, not absolute predictive accuracy in operation. Four further bounds apply, and we state them rather than leave them implicit.

*The two oracles agree weakly.*  $I^*(v)$  and  $I_{\text{comp}}(v)$  correlate at mean  $\rho = 0.394$  (Section 7.5). Results established against one do not transfer to claims measured against the other, which constrains this paper’s own internal cross-referencing: Section 5.4’s library finding and Section 5.5’s stratified check are  $I_{\text{comp}}$  results and are not evidence about the  $I^*$ -backed tables in Section 8.1.

*Six instrument defects were found and corrected during this revision.* All were silent — none raised an exception or produced an obviously wrong number — and all are recorded here because each had, or could have had, a published figure resting on it. The first two predate the corpus regeneration of Section 7.1. First,



the **Topo-QoS** baseline was applying no QoS weighting:  $w(t)$  is declared on the Topic node, the harness looked for it on the pub-sub relationship, and the generated topologies carry none there, so every derived dependency edge kept a unit weight and the baseline computed plain betweenness on all seven scenarios. It has been repaired to resolve  $w(t)$  from the shared Topic; the affected columns of Tables 18 and 20 and the k-fold table were recomputed, and the non-QoS variants were verified unchanged to machine precision. Second, HGT attention extraction captured nothing, because HGTConv in the pinned PyTorch Geometric release exposes no `return_attention_weights` argument and the extraction fell through its own error branch; attention is now captured from the layer’s own softmax, and the attention subgraph of Fig. 3 is generated from real per-edge  $\alpha$  rather than an edge-weight fallback. We note that the second defect had masked a third — the subgraph renderer itself raised on a `networkx` API change, which nothing had exercised while the attention payload was empty.

Four further defects were found in a later pass that checked the implementation against this manuscript directly, rather than against a specific reported number.

Third, and the one that changes reported figures, **FaultInjector**’s cascade iterated an unordered Python set of subscribers while consuming seeded random draws for each; set iteration order in Python is salted per-process by `PYTHONHASHSEED`, so the *same* requested seed could assign different draws to different subscribers across processes, making  $I^*(v)$  reproducible only within one interpreter run, not across runs — exactly the kind of instability the seed-mean-and-standard-deviation protocol of Section 5.1 and Section 7.5 was designed to average over, not diagnose. It has been repaired (the iteration is now sorted); cross-process reproducibility was verified directly (identical  $I^*(v)$  across five different `PYTHONHASHSEED` values, both on the synthetic corpus and on the real-world Autoware sweep of Section 8.5). Two figures rest on the pre-fix labels and are flagged rather than silently carried forward: the label test–retest  $\rho$ /Jaccard ceiling of Section 7.5, restated above with the corrected, now process-independent values (previously reported as  $\rho \in [0.928, 1.000]$ , Jaccard  $\in [0.56, 1.00]$ , both measured within a single process and so blind to this defect); and Section 8.5’s Autoware row, whose “sweep-to-sweep instability” was reported in an earlier draft as a property of that graph and is corrected there to what it actually was. Tables 18 and 20, and every scenario in Table 13, are unaffected: both use `cascade_depth_limit=0`, the setting at which the sixth defect below is provably a no-op, and neither exercises the code path this defect lived in independently of that setting.

Fourth, `extract_rmav_scores_dict` — the function that turns `PredictionService`’s RMAV output into the GNN’s auxiliary training target — keyed its lookup by an attribute (`component_id`) that the underlying dataclass does not have (it has `id`), so every key fell through to the object’s own `repr()` string and the lookup silently returned nothing usable; the 0.1-weighted RMAV-consistency term of Table 17 was training against an all-zero target wherever this function was on the path. It has been repaired to key by `id` first, matching its sibling function’s already-correct convention. Table 3/5/6/7’s reported runs are unaffected: both evaluation harnesses (`cli/loso_evaluate.py`, `cli/kfold_evaluate.py`) read RMAV scores through

a different loader that never called the broken function. Any GNN checkpoint trained via the standalone `cli/train_graph.py` entry point without an explicit `--rmav` file did go through the broken path and trained with no RMAV supervision; that entry point is not what produced the tables in this paper.

Fifth, the parallel worker in the prescription stage’s per-edit verifier (Section 6.4) constructed its own evaluator with default settings — layer `system`, no GNN checkpoint — regardless of what layer and checkpoint the run was actually configured with, so a `--layer app` run would score every candidate edit’s counterfactual impact on the `system` layer while its baselines were measured on `app`. It has been repaired to thread the configured layer and checkpoint into each worker. Table 13 is unaffected: `reproduce/run_prescribe_all.py` runs at `layer="system"` with no checkpoint, which is exactly what the unpatched default constructed, so the mismatch could not occur for the reported run.

Sixth, the post-loop computation of  $I^*(v)$  read a `topic_loss` variable left over from the cascade’s last executed wave rather than recomputing it against the final set of failed components, so a subscriber failure in the final wave was not reflected in that subscriber’s own reported feed loss. It has been repaired to recompute once more after the loop terminates. This defect is a no-op when `cascade_depth_limit=0` (unlimited waves, the default and the setting `reproduce/` and the corpus generation in Section 7.1 both use throughout): we verified this directly by running the pre-fix and post-fix simulators against the same cached topologies and confirming a maximum absolute difference in  $I^*(v)$  of exactly 0 across three scenarios. It is not a no-op under a finite `cascade_depth_limit`, which no reported figure in this paper uses.

*A third of each system is unlabelled.* The cascade model cannot express the failure of a Topic or a physical Node, leaving 30–47% of components per scenario without ground truth. Predictions for them are produced but never validated. Broker labels are degenerate in three of seven scenarios for a related reason. Any claim of coverage across “all five component types” would be unsupported, and the per-type results report those strata as undefined rather than as zero.

*Reported figures approach the labels’ own reproducibility.* The ground truth agrees with itself at test–retest  $\rho$  of 0.807–1.000 and top- $K$  Jaccard of 0.44–1.00 across seeds (post the determinism fix above; these are now stable across `PYTHONHASHSEED`, unlike the figures an earlier draft reported). A model scoring near the former has saturated the labels rather than underperformed, and every top- $K$  metric inherits the latter’s churn.

*The behavioural oracle is delivery-based, not QoS-aware.*  $I_{\text{dyn}}$  carries the construct-validity argument of Section 7.5, so the limits of what it measures bound that argument too. Its discrete-event engine implements deadline, lifespan, and reliability enforcement, but resolves topic QoS from an attribute key the generated corpus does not write, so every run in this evaluation falls back to defaults and the deadline and best-effort drop paths are structurally zero rather than measured as zero. Latency is likewise uninformative here: at the corpus’s publication rates, utilisation stays far below saturation and queues never build, leaving  $p95$  latency flat to within run-to-run jitter across faulted components.  $I_{\text{dyn}}$  should therefore be read as a *throughput* oracle — it corroborates that the cascade ranking tracks lost message delivery, and it makes no claim about

QoS contract conformance under load.

**Internal validity.** The chief internal risk is circular validation — a predictor scoring well because its inputs leaked from its labels. The framework addresses this by *view* separation: predictors operate on  $G_{\text{analysis}}$  while ground truth is generated by simulating  $G_{\text{structural}}$ , no simulation output is fed back as a predictor feature, and remediation candidates are generated without reading simulated impact. **This is view independence, not independence of data source:** both views are deterministic functions of the same input topology, so what is ruled out is feature–label feedback, not the possibility that both encode a shared modelling assumption. The distinction matters for how much weight the guarantee can bear, and we prefer to state it than to let “independent simulator” imply more.

The behavioural oracle narrows this, and it is worth being precise about by how much. A sharper form of the circularity objection is that  $I^*$  is an artifact of its own traversal — that a topology-derived score is being validated against labels manufactured by walking the same topology.  $I_{\text{dyn}}$  answers that specific charge: it reaches its ranking by simulating message traffic through queues over simulated time, never traversing `DEPENDS_ON`, and it recovers  $I^*$ ’s ordering (Section 7.5). The cascade *algorithm* is therefore not the artifact. What remains unaddressed is the layer beneath it: all three oracles are simulation rather than observed failure data, and all three are deterministic functions of the same generated topology. A modelling assumption shared by the architecture model itself would be invisible to every one of them. Calibration against instrumented deployments (Section 9.3) is the only thing that reaches it.

Two further internal-validity issues surfaced during a pre-submission audit of this work and are disclosed because they invalidated previously reported numbers. First, the evaluation harness scored different predictor families on different node populations and different samples (Section 7.3); the correction changed the sign of the RQ1 conclusion. Second, the Leave-One-Scenario-Out sweep reused stale model checkpoints and was therefore not training at all — the same command produces  $\rho = -0.576$  in 3.2 s against a dirty workspace and  $\rho = +0.594$  in 322 s against a clean one. Both are fixed and all reported figures come from the corrected runs, but the episode is itself a finding about this class of experiment: a silently-cached artifact is indistinguishable from a trained one in the output, and only the implausible wall-clock time exposed it.

*Artifact retention is uneven across the reported tables, and one headline table cannot currently be regenerated.* Table 18 and the sensitivity sweeps of Section 8.3 regenerate exactly from stored result files — a claim that held only approximately before the determinism defect above was fixed, since a re-run in a fresh process was not guaranteed to reproduce a stored `FaultInjector` label exactly even at an unchanged seed. It now holds without qualification. The Leave-One-Scenario-Out result file behind Table 20 does not exist: it was overwritten during the revision, and the most recent retained log for that sweep predates the baseline repair and records a different ordering (Section 8.1). We disclose this rather than present Table 20 on the same footing as Table 18, and we regard it as the direct continuation of the two defects above. The common mechanism in all three is that an experiment’s *evidence* and its *output* were allowed to come apart — a

cached checkpoint, a mismatched sample, an unretained result file — and in each case the number looked entirely ordinary. The discipline this study now imposes, and did not impose soon enough, is that no figure enters the manuscript unless the artifact that produced it is retained and the figure can be recomputed from it. Table 20 is the outstanding exception, and re-running it under the final apparatus is the first item of remaining work.

**External validity.** This is the weakest dimension of the study, and we regard it as the highest-value follow-up (Section 9.3).

*The corpus spans ten deployment domains, but only three architectures are not ours.* Seven scenarios come from a single statistical topology generator; the three real-world graphs (Autoware.universe ROS 2, the Cloud-Native Microservices mesh, and Train-Ticket) are transcribed from published open-source architectures. On the latter, SaG achieves mean rank correlation over five seeds of  $\rho = 0.688, 0.778$  and  $0.759$ , and up to  $F_1@K = 1.000$  on two of the three.

*None of the three clears the framework’s own gate.* All three fail SPOF-F1; Autoware additionally fails the  $\rho$  threshold and Cloud Microservices the predictive-gain threshold, for 5 failed checks of 15 (Section 8.5). The  $F_1@K = 1.000$  figures are partly a tie-breaking artifact of  $K$  exceeding the count of genuinely non-zero-impact components. We read the result as evidence that the predictive *ranking* transfers beyond the generator to independently-sourced architectures — not as a demonstration of production readiness.

*The paradigm count is two, not three.* Train-Ticket and Cloud Microservices are both microservice meshes, so cyber-physical pub-sub is represented by Autoware alone. Leave-One-Scenario-Out evaluation confirms inductive transfer across held-out *synthetic* architectures only, since all seven share a generator. Expanding to further middleware paradigms and to hardware-in-the-loop deployments is future work (Section 9.3).

**Conclusion validity.** Criticality scores and simulated impact metrics exhibit heavy-tailed, non-parametric distributions that violate normality assumptions. To prevent classification bias, we apply non-parametric rank correlations (Spearman  $\rho$ ), top- $K$  Jaccard metrics, and adaptive box-plot thresholding ( $Q3 + 1.5 \text{ IQR}$ ) rather than parametric z-scores or arbitrary absolute cutoffs (Section 4.4).

### 9.3. Limitations and Future Work

Several limitations point to concrete next steps, ordered here by how much they would change the paper’s claims.

**Real-world deployment validation and HIL execution.** Section 8.5 narrows the external-validity gap on three real-world open-source software architectures (Autoware.universe ROS 2, the Cloud-Native Microservices Mesh, and Train-Ticket), but does not close it: the graphs are hand-transcribed, their ground truth is still simulated, and none clears the framework’s own validation gate in full. Validating predictions against runtime hardware-in-the-loop (HIL) fault injection on physical testbeds, and against harvested rather than transcribed architectures, remains the highest-value follow-up.

**Out-of-distribution ranking is not yet a solved problem.** Section 8.1 shows that a training-free QoS-weighted centrality matches the learned models on LOSO rank correlation, with the learned advantage confined to critical-set identification. Whether that ceiling reflects the difficulty of cross-architecture transfer, the noise floor of the labels (Section 7.5), or a limitation of the architecture is not resolved by these experiments. Distinguishing those three explanations — plausibly by training on a substantially larger and more diverse scenario corpus — would determine whether typed learning has more to offer here than it currently demonstrates.

**The dimension weighting does not improve accuracy.** Section 8.3 finds equal weights outperform the calibrated AHP weighting with no plateau in the shrinkage parameter. We have repositioned RMAV as an attribution mechanism accordingly, but a weighting *derived* rather than asserted — fitted to simulated impact, or elicited from a panel of practitioners with reported inter-rater agreement — would let the decomposition make an accuracy claim as well as an explanatory one.

**The Vulnerability dimension is the lightest of the four,** resting on reachability-style proxies with no model of trust boundaries, privilege, or data sensitivity. A richer adversarial model would strengthen the V attribution and broaden the framework’s security relevance.

**Remediation is verified but not yet demonstrably effective.** The per-edit acceptance filter of Section 6.4 is implemented, which removes the possibility of an unverified regressing edit being applied (Section 6.7). What it does not do is make the operators work: the risk reductions it achieves are real but small (+0.0025 to +0.0158 SRI), and the operator set is narrow enough that its yield depends heavily on whether the topology happens to contain a fan-out bottleneck. Expanding the operator set — along with deriving the acceptance multiplier  $\kappa$  from broader multi-seed variance data rather than fixing it at 1.0 — is needed before the prescriptive stage can claim practical value. One specific gap follows: verification currently admits *singletons*, each simulated on a graph containing that edit alone, so verifying subsets rather than single edits is required before an accepted policy can be called compositionally safe. That is engineering work rather than an open research question, and we flag it as the immediate next step for this stage.

**The CI/CD gate is absolute, not yet delta-aware.** Section 6.6 implements exit-code gating on a candidate topology’s full finding set; it does not yet diff that set against a merge-base topology, so an architecture carrying an intentional, previously-accepted risk (a sole-source feed, a deliberately unreplicated legacy broker) fails the build on every commit rather than only when a change introduces something new. A delta-aware gate — evaluate candidate and merge-base topologies, block only on findings absent from the baseline — together with a waiver register recording accepted risk (entity, rule, expiry) so it stays auditable rather than silently re-suppressed on every run, is the fix; neither is implemented today, and closing this gap is engineering work rather than an open research question, similar in kind to the remediation gap above. A related, smaller gap is that `detect_antipatterns.py` itself requires a live Neo4j connection even though the underlying analysis machinery is already usable through the database-free `MemoryRepository` — wiring that

path into the packaged CLI is a prerequisite for running the gate without a database in an actual CI job.

**Edge-level ground truth is bounded by the cascade model.** Edge criticality is now measured by removal rather than inferred from endpoints (Section 8.2), but the cascade routes no traffic over `RUNS_ON` or `CONNECTS_TO` relations, so those edges measure as exactly zero regardless of their structural role. Extending the cascade to express infrastructure-layer failure would close the same gap that leaves Topic and physical Node components unlabelled.

**Relationship attribution is defined but not validated, and closing that gap is out of scope for this submission.** Section 4.7 gives D2 a measure with the same signature as D1, implemented and computed on every derived dependency. What it does not have — and, as the framework is currently constructed, cannot have — is the correlation-style evidence Section 8.1 supplies for nodes: attribution is scored on `DEPENDS_ON` edges while the removal oracle severs raw structural edges, so the two are never defined on a common population (Section 4.7). Re-simulating directly on the derived graph is not an available fix, since the independence guarantee (Section 5.3) requires the simulator to operate only on  $G_{\text{structural}}$ . The one route that respects that guarantee is to track, for each derived edge, which raw edges mediate it, and aggregate their measured impact onto it — but the mediating relations are many-to-many, so this requires a real modelling decision (how to aggregate) rather than a mechanical lift, and we leave it for future work rather than attempt it here. Until it is done, the relationship half of the framework rests on construction rather than on measurement, which we regard as the most significant open gap in the diagnostic path.

**Finally, the endpoint for all of the above** is calibration against observed failure data from instrumented deployments, which would convert this paper’s comparative claims into absolute ones.

#### 9.4. Conclusion

We presented Software-as-a-Graph, a pre-deployment Static System Analysis (SSA) framework that models distributed pub-sub middleware as a typed, weighted, directed multigraph and analyzes it along two coupled axes: multi-dimensional quality attribution, which decomposes each component’s criticality into orthogonal, interpretable RMAV dimensions (integrating local code quality metrics), and failure-impact analysis, which predicts cascade impact with both the interpretable composite and a learned heterogeneous graph transformer, validated against discrete-event simulation under a strict input-label independence guarantee. A prescriptive remediation stage generates topology-level hardening edits from structure alone and verifies each one individually against the canonical simulator, admitting it only when its benefit exceeds the simulator’s own seed noise at every propagation threshold; under that filter 162 of 332 candidates survive across the seven scenarios, but the resulting risk reductions are small (+0.0025 to +0.0158 SRI) and concentrated in the two topologies with pronounced fan-out structure, which we report as the substantive — and qualified — result of that stage (Section 6.4, Section 6.7).

Integrated directly into pipelines as a delta-aware, blocking CI/CD Quality Gate, the framework verifies

architectural changes and blocks regression in seconds, bridging the “Architecture-Code Gap” at commit time. Across a synthetic scenario suite, the framework establishes a scope condition on where typed graph learning pays — it leads a training-free QoS-weighted centrality on both ranking ( $\rho = 0.608$  vs 0.521 out of distribution) and identifying *which* components belong on a shortlist ( $F_1@K = 0.465$  vs 0.308), both measured only after repairing that baseline, which had been computing no QoS weighting at all. Alongside that, measuring edge criticality by removal rather than inferring it from endpoints shows most individual links to be replaceable and exposes a class of relations the cascade model cannot express at all, and stratified rather than pooled reporting caught a distortion that moved a headline figure by 0.38. By taking the *type* of every component and dependency seriously, the framework recovers structure that untyped, single-dimensional methods discard, and does so at the point in the lifecycle where it is most valuable: before the system runs.

---

## CRediT authorship contribution statement

*[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]*

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Funding

*[Omitted for double-anonymised review.]*

## Data availability

The seven synthetic scenario datasets, their generator configurations, and the manifest of canonical dataset hashes are included in the replication package, from which every synthetic dataset regenerates byte-identically. The three real-world architecture graphs of Section 7.1 and their adapter are included on the same terms. Result artifacts are provided for the in-distribution evaluation (Table 18), the sensitivity sweeps of Section 8.3, the edge-removal measurement of Section 8.2, the remediation sweep of Section 6.7, and the three real-world validation runs of Section 8.5. **One exception is recorded rather than glossed:** the per-fold artifact behind the Leave-One-Scenario-Out results of Table 20 was not retained and cannot be regenerated from the archive without re-running the sweep; Section 8.1 and Section 9.2 state what follows from that. A link to the archived package will be supplied on acceptance.

## Declaration of generative AI use

*[To be completed by the authors in accordance with the journal's policy.]*

## References

- [1] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [2] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [3] OASIS, MQTT version 5.0, OASIS Standard (2019).
- [4] [Anon-A], Authors' prior work on multi-layer graph dependency analysis for publish–subscribe systems, citation withheld for double-anonymised review (n.d.).
- [5] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [7] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering* SE-2 (4) (1976) 308–320.
- [8] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [9] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [10] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [11] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [12] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [13] SonarSource, Clean as you code, SonarQube documentation (2024).
- [14] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.



- [15] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [16] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [17] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [18] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [19] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [20] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [21] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [22] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [23] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [24] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [25] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [27] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [28] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.

- [29] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [30] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
- [31] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: Proc. AAAI Conference on Artificial Intelligence, 2018, pp. 3538–3545.
- [32] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, X. Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: Proc. AAAI Conference on Artificial Intelligence, 2020, pp. 3438–3445.
- [33] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [34] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [35] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.
- [36] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.
- [37] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.
- [38] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [39] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [40] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.
- [41] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.

- [42] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [43] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [44] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.
- [45] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [46] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
- [47] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: *Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296.
- [48] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, *Software artifact* (2024).
- [49] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [50] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [51] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
- [52] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [53] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.